

2

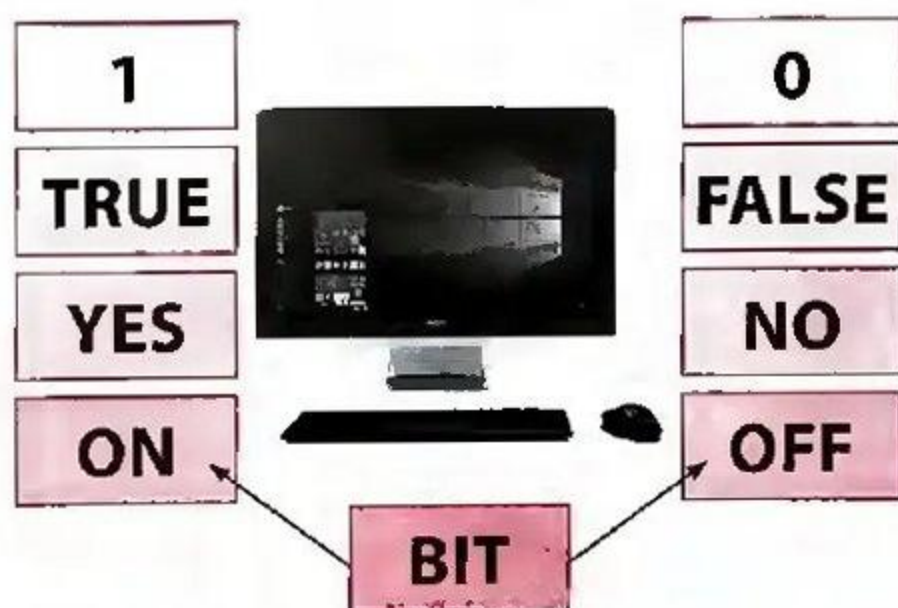
Data Representation and Boolean Logic

2.1 INTRODUCTION

You must have heard that computers operate in binary, meaning they store data and perform calculations using only ones and zeros, *i.e.*, ones and zeros play a big role in how computers work. In computers, the user input is first converted and transmitted as electrical pulses that can be represented by two unique values, ON and OFF. The 'ON' state may be represented by '1' and the 'OFF' state by '0'. The sequence of ONs and OFFs forms electrical signals that the computer can understand. In other words, data inside a computer gets stored and manipulated in digital (binary) form (0s and 1s).

Inside a computer, it is all about wires and circuits which carry all the information. Now, the question arises as to how do you store or represent information using electricity?

If it is a single wire, there exist two states—ON/OFF; Yes/No; True/False; 1/0.



This ON and OFF together constitute a BIT which is the smallest piece of information a computer can store as shown in Fig. 2.1.

Thus, if we use more wires, we get more bits with more 0s and 1s; with more bits, we can represent more complex information as shown in the given figure.

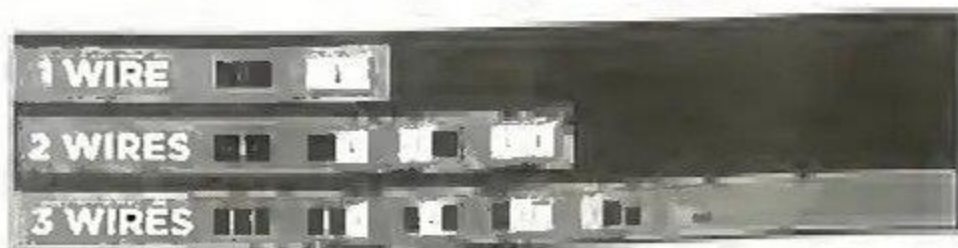


Fig. 2.1: Data Representation inside a Computer

Different data stored in the computer may be of the following kinds:

- Numeric data (0, 1, 2,, 9)
- Alphabetic data (A, B, C,, Z and a, b, c,, z)
- Alphanumeric data—a combination of any of the symbols (A, B, C,, Z), (a, b, c,, z) (0, 1,, 9)
- Special characters (+, -, #, @, \$, %, ?, Blank), etc.
- Multimedia data such as audio, video, graphics, images, etc.

These kinds of data, be it alphabets, numbers, symbols, sound data or video data, are represented in terms of 0s and 1s, in the computer. Each symbol is represented as a unique combination of 0s and 1s.

Digital computers represent data by means of an easily identifiable symbol called a digit. These digits can be represented in different types of number systems used in digital technology. This chapter deals with the types of number systems and their conversion from one form to another.

Learning Tip: A microprocessor (CPU) is a small electronic component inside a computer built on a single chip called integrated circuit (IC) that performs basic arithmetic and logical operations on data. An integrated circuit is one piece of semiconductor material loaded with transistors and other electronic components. A transistor is a tiny switch that is activated by the electronic signals it receives. The digits 1 and 0 used in binary reflect the on and off states of a transistor.

2.2 NUMBER SYSTEM

Number systems are the technique to represent numbers in the computer system architecture. Every number system includes a set of unique characters or literals. It is a way to represent a number in different forms. When this data is fed into the computer, it is converted into numeric format according to ASCII (American Standard Code for Information Interchange), where each type of data is represented by its numerical equivalent. This encoded value is further converted to its equivalent binary representation so that it is understandable by the computer.

In general, we are familiar with the popular number system known as decimal number system in which 10 digits, namely 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9 are used to represent any number.

Similarly, in the field of computers, we deal with two binary digits known as 1 and 0 that represent the presence or absence of an electric pulse respectively. There are infinite ways to represent a number.

DECIMAL

0	1	2	3	4
5	6	7	8	9

BINARY

0	1
---	---

Each digit in a number system depends upon the following:

- (i) The face value of the digit, *i.e.*, the digit itself.
- (ii) The base of the number system, also called radix.
- (iii) The position of the digit in the number.

The four commonly associated number systems with modern computers and digital electronics are: decimal, binary, octal and hexadecimal.

CTM: Number systems are the technique to represent numbers in the computer system architecture.

In digital representation, the number systems are classified into four fundamental types:

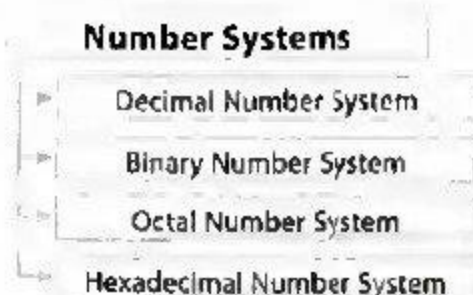


Fig. 2.2: Classification of Number System

➤ Decimal Number System

Decimal number system is the most widely used number system in computer applications. This number system has ten (10) digits ranging from 0 to 9 with base value 10. Every number (value) can be represented with 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9 in this number system. Each value in this number system has the positional value expressed as a power of 10. For example, in the decimal number 6945, the value 6 at position number 3 with base 10 has positional value 10^3 .

The representation of a decimal number with fractional part is depicted in Fig. 2.3(a).

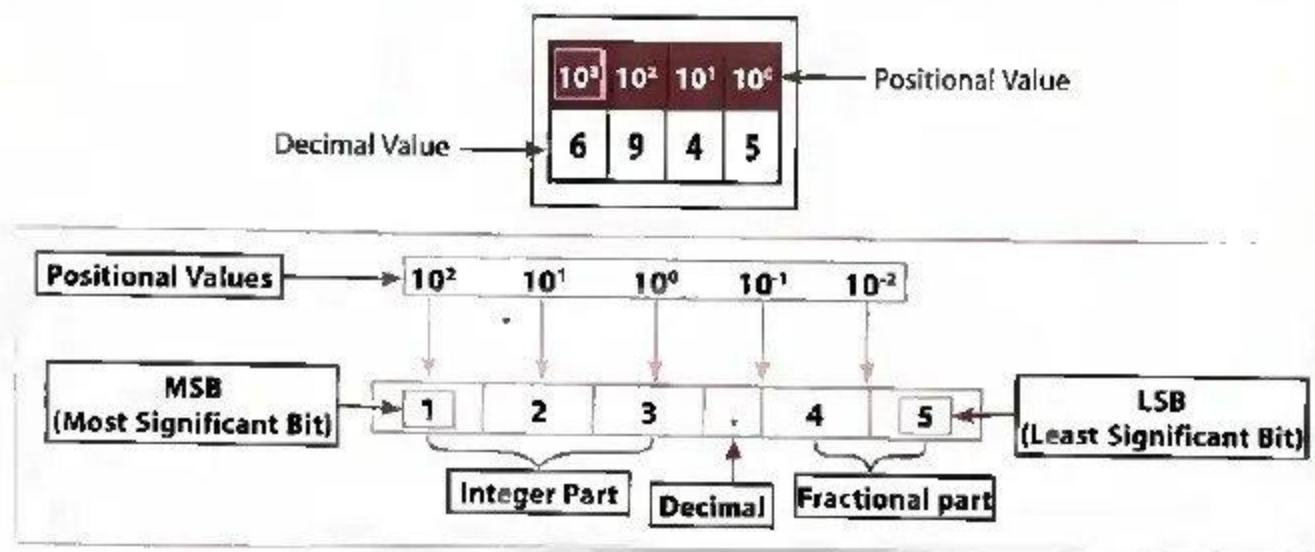


Fig. 2.3(a): Representation of Decimal Number System

Here, in the decimal number with fractional part 123.45, the positional values to the left of the decimal are positive powers of 10 (starting from 10^0) and increase from right to left by 1. On the other hand, in the fractional part of the number, the positional values to the right of the decimal are negative powers of 10 (starting from 10^{-1}) and decrease by 1 from left to right.

➤ Binary Number System

One computer chip can have millions of transistors activated by the electronic signals and continually switching between ON and OFF which is represented using two digits, 1 and 0, respectively. These digits are called binary number or machine code.

A binary number system has only two digits—0 and 1.

Every number (value) can be represented with 0 and 1 in this number system. The base of binary number system is 2 because it has only two digits. Each digit in this number system also has the positional value expressed as a power of 2. The Pictorial representation of a binary number $(10101.1101)_2$ is depicted in Fig. 2.3(b).

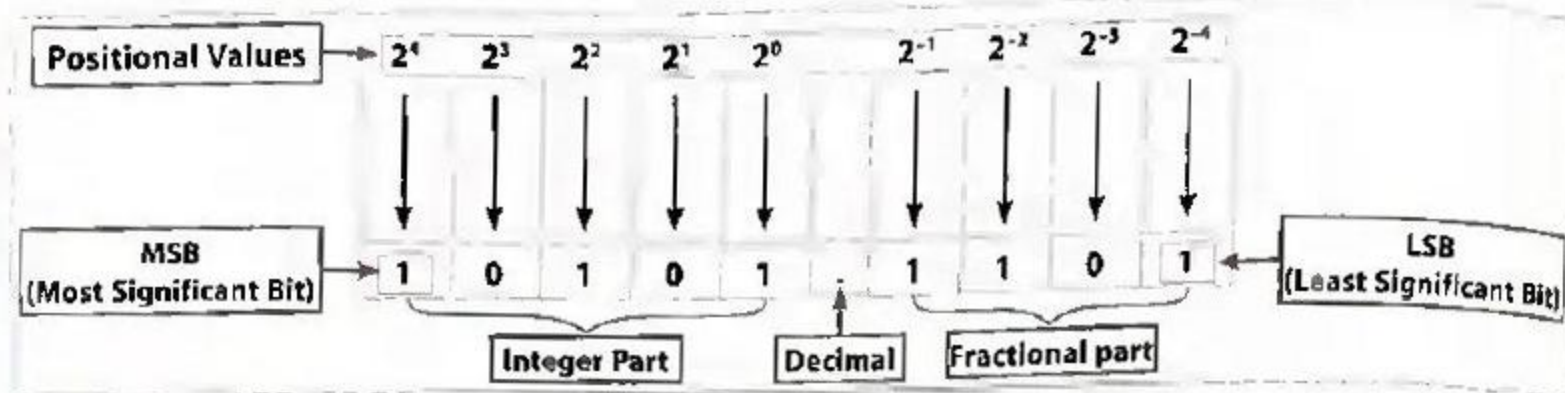


Fig. 2.3(b): Representation of Binary Number System

Here, the positional values to the left of the decimal are positive powers of 2 (starting from 2^0) and increase from right to left by 1. On the other hand, places to the right of the decimal are negative powers of 2 (starting from 2^{-1}) and decrease by 1 from left to right.

➤ Octal Number System

Octal number system has only eight (8) digits from 0 to 7. Every number (value) can be represented with 0, 1, 2, 3, 4, 5, 6 and 7 in this number system. The base of octal number system is 8 because it has only 8 digits.

The pictorial representation of an octal number with fractional part is depicted in Fig 2.3(c).

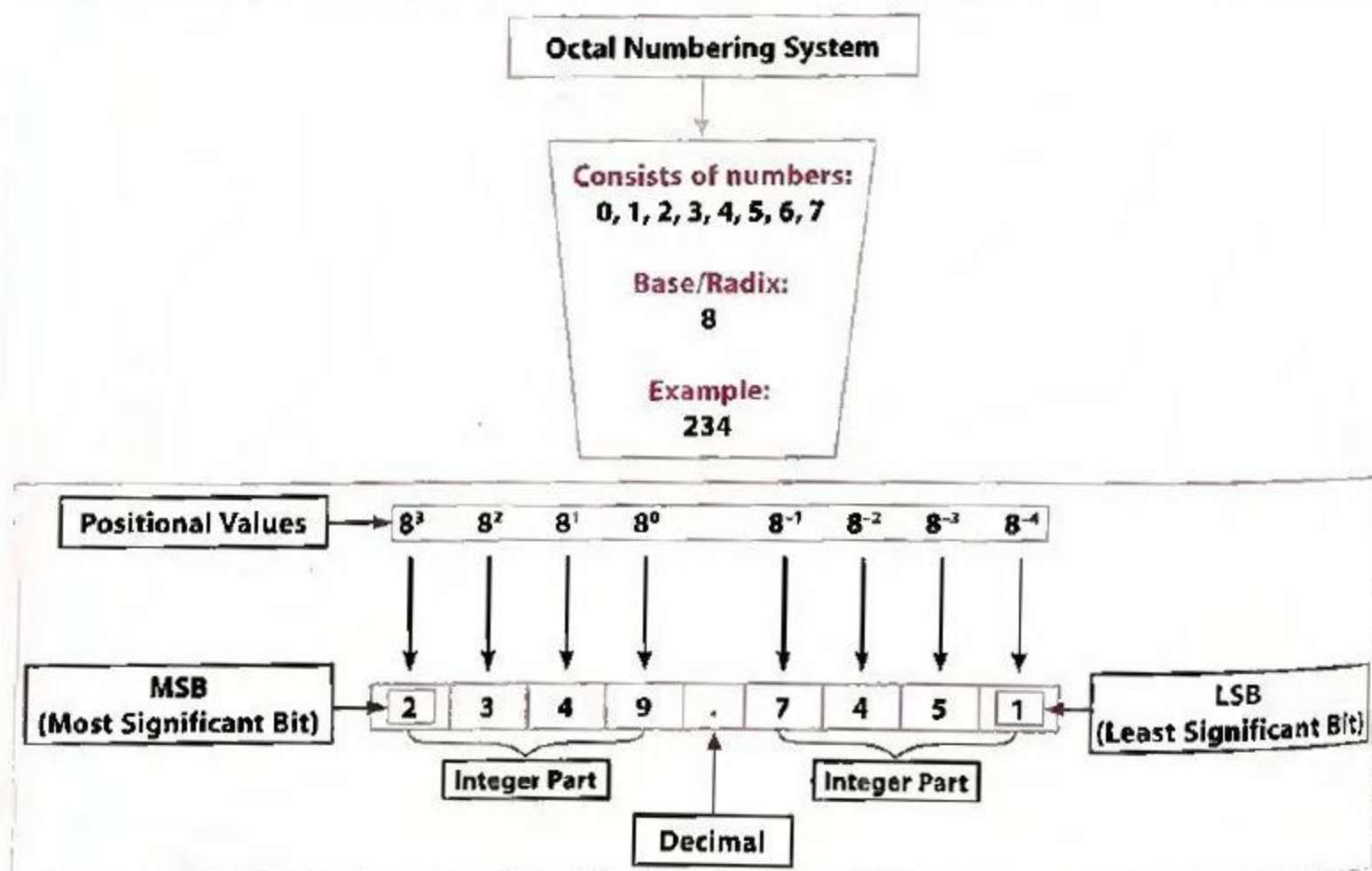


Fig. 2.3(c): Octal Number System

Here, the positional values to the left of the decimal are positive powers of 8 (starting from 8^0) and increase from right to left by 1. On the other hand, places to the right of the decimal are negative powers of 8 (starting from 8^{-1}) and decrease by 1 from left to right.

➤ Hexadecimal Number System

A hexadecimal number system has sixteen (16) alphanumeric values from 0 to 9 and A to F. Every number (value) can be represented with 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E and F in this number system. The base of hexadecimal number system is 16 because it has 16 alphanumeric values. Here, the decimal numbers 10 through 15 are represented by letters A through F, i.e., A is 10, B is 11, C is 12, D is 13, E is 14 and F is 15.

Pictorially, hexadecimal number $(2A3.45)_{16}$ can be represented as:

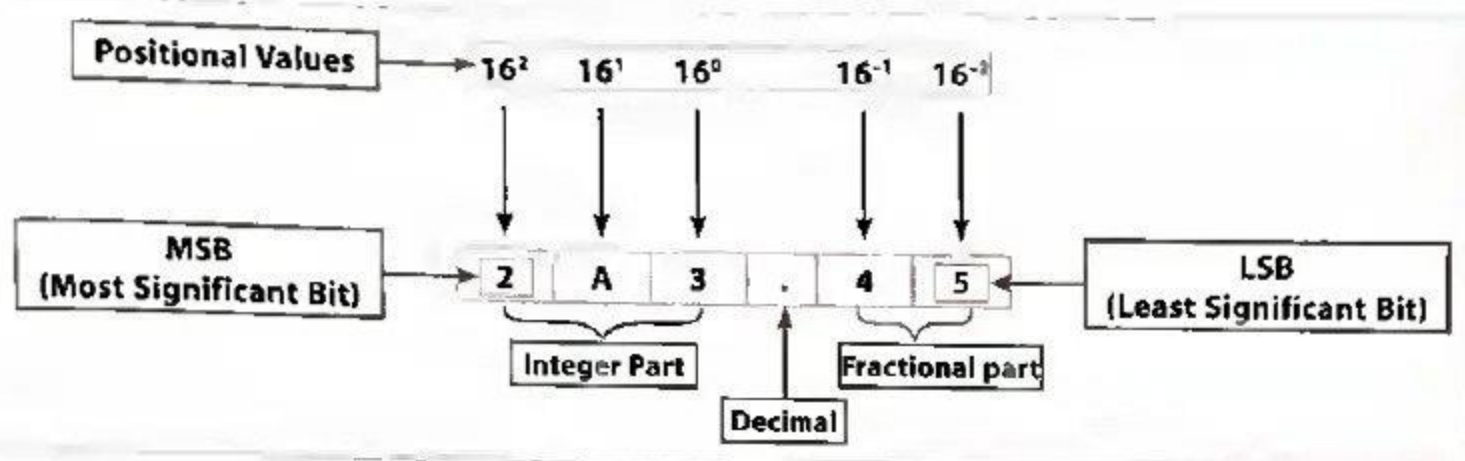


Fig. 2.3(d): Representation of Hexadecimal Number System

Here, the positional values to the left of the decimal are positive powers of 16 (starting from 16^0) and increase from right to left by 1. On the other hand, places to the right of the decimal are negative powers of 16 (starting from 16^{-1}) and decrease by 1 from left to right.

Each number system has a base known as a Radix. A decimal number system is a system of base 10; binary is a system of base 2; octal is a system of base 8; and hexadecimal is a system of base 16. What are these varying bases? The answer lies in what happens when we count up to the maximum number that the numbering system allows. In base 10, we can count from 0 to 9, that is, 10 digits.

Table 2.1: Number Systems and their Symbols

Number System	Base	Symbols used
Binary	2	0, 1
Octal	8	0, 1, 2, 3, 4, 5, 6, 7
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Hexadecimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F where A = 10; B = 11; C = 12; D = 13; E = 14; F = 15

CTM: (1) The left-most bit of a number having the greatest value is known as Most Significant Bit (MSB).
(2) The right-most bit of a number having the lowest value is known as Least Significant Bit (LSB).
It is same for all number systems.

Applications of hexadecimal numbers:

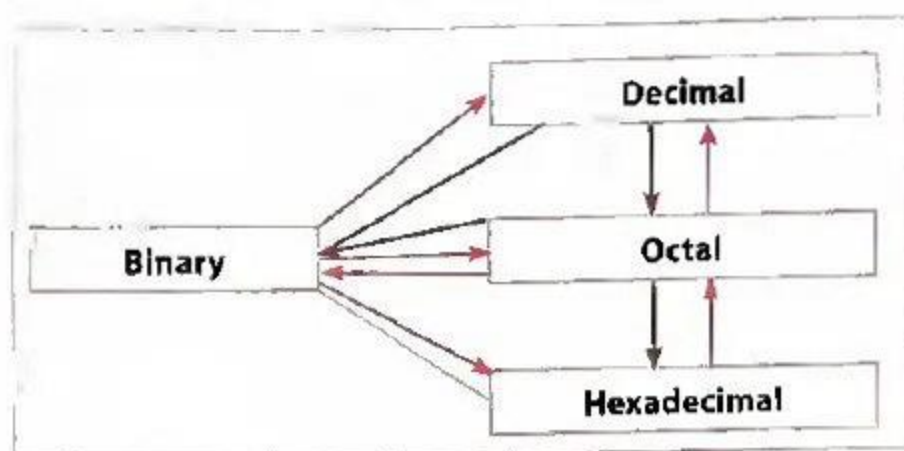
- Hexadecimal number system is used to specify the memory location with unique address that is either 16-bit or 32-bit long, which is difficult to deal with. For example, a 16-bit memory address 1111000010101111 is mapped to FOAF using hexadecimal notation which is easier to remember.

- Hexadecimal number system is also used for describing the colour codes on the web pages or image-editing programs using the format #RRGGBB. The first two letters/numbers refer to red, the next two refer to green, and the last two refer to blue. The colour values are defined in values between 00 and FF. For example, #FF0000 will generate red colour.

2.3 NUMBER SYSTEM CONVERSIONS

In the previous topic, we have discussed the various number systems available to us for saving and retrieving the values stored inside a computer system. Normally, we use base 10 or decimal number system in our day-to-day life. But when computers came into operation, decimal representation was changed to binary system as it involves binary computation.

Since binary involves a long chain of BITS which is difficult to remember, Octal and Hexadecimal systems were developed as both these systems are much convenient and, hence, conversion between these number systems became mandatory which is being discussed below. Conversion of one number system to another is classified into the following three broader categories:



Decimal Number System to Other Base

- Decimal Number System to Binary Number System
- Decimal Number System to Octal Number System
- Decimal Number System to Hexadecimal Number System

Other Base to Decimal Number System

- Binary Number System to Decimal Number System
- Octal Number System to Decimal Number System
- Hexadecimal Number System to Decimal Number System

One Base to Other Base System

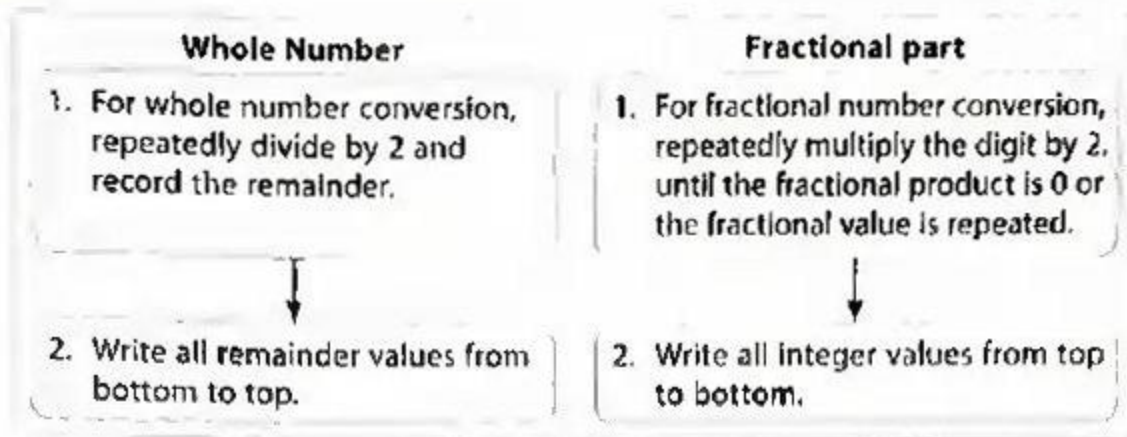
- Binary Number System to Hexadecimal Number System
- Hexadecimal Number System to Binary Number System
- Binary Number System to Octal Number System
- Octal Number System to Binary Number System
- Hexadecimal Number System to Octal Number System
- Octal Number System to Hexadecimal Number System

Let us discuss these in detail.

2.3.1 Decimal Number System to Other Base

Decimal number to Binary number

Following steps are to be carried out to convert a Decimal number into its Binary equivalent:



- Step 1:** Divide the given decimal number by 2 and record its quotient Q_1 and remainder R_1 .
- Step 2:** Again divide the new quotient obtained from Step 1 by 2 and record the new quotient Q_2 and remainder R_2 .
- Step 3:** Repeat Step 1 and Step 2 until the quotient Q_n becomes 0.
- Step 4:** Write all remainder values R_n moving from bottom to top, i.e., moving from most significant bit (MSB) to least significant bit (LSB).

Example 1:

- Convert the Decimal number 125 into its Binary equivalent.

2	125	Remainder
2	62	1
2	31	0
2	15	1
2	7	1
2	3	1
2	1	1
	0	1

LSB
↑
MSB

Learning Tip: To convert decimal number into other number system, we write all remainder values moving from bottom to top.

Answer: $(1111101)_2$

Example 2:

- Convert $(95)_{10} = ()_2$

2	95	Remainder
2	47	1
2	23	1
2	11	1
2	5	1
2	2	1
2	1	0
	0	1

LSB
↑
MSB

Answer: $(95)_{10} = (1011111)_2$

Converting Decimal fraction Into Binary fraction

While converting decimal fraction into binary fraction, the procedure is to successively multiply the decimal fraction by the radix, i.e., 2, and follow the given algorithm.

Step 1: Multiply successively each fractional value by 2.

Step 2: Write separately the integral part, which is to the left of the decimal point, and the fractional part, which is to the right of the decimal point.

Step 3: Multiply the fractional part again by 2 and record the new integral and fractional part. This procedure will continue until either the fractional part becomes 0 or is repeated.

Step 4: Moving from top to bottom, write all values of integers.

Example 3:

Convert $(0.375)_{10}$ into binary fraction.

Step 1: Multiply 0.375 by 2. We get

$$0.375 \times 2 = 0.750 \quad \begin{array}{cc} \text{Integer} & \text{Fraction} \\ 0 & 0.750 \end{array}$$

Step 2: Multiply 0.750 again by 2. We get

$$0.750 \times 2 = 1.50 \quad \begin{array}{cc} \text{Integer} & \text{Fraction} \\ 1 & 0.50 \end{array}$$

Step 3: Repeat this step until fractional part becomes 0 or is repeated.

$$0.50 \times 2 = 1.0 \quad \begin{array}{cc} \text{Integer} & \text{Fraction} \\ 1 & 0.0 \end{array} \quad \text{Fractional Part is 0}$$

Step 4: Stop multiplying fractional part by 2 as it is now 0.00.

Move from top to bottom and write all integer values (from top to bottom), the result is $(0.375)_{10} = (0.011)_2$

Example 4:

Convert $(105.15)_{10}$ into binary.

Let us convert 105 first as shown below.

$$(105)_{10} = (1101001)_2$$

2	105	Remainder
2	52	1
2	26	0
2	13	0
2	6	1
2	3	0
2	1	1
	0	1

LSB

↑

MSB

$$\text{Answer: } (105)_{10} = (1101001)_2$$

Let us convert $(0.15)_{10}$

	Integer	MSB
Multiply 0.15 by 2	0.30	↓
Multiply 0.30 by 2	0.60	
Multiply 0.60 by 2	1.20	
Multiply 0.20 by 2	0.40	
Multiply 0.40 by 2	0.80	
Multiply 0.80 by 2	1.60	
		LSB

Learning Tip: To convert fractional decimal number to other number system, we write all integer values moving from top to bottom.

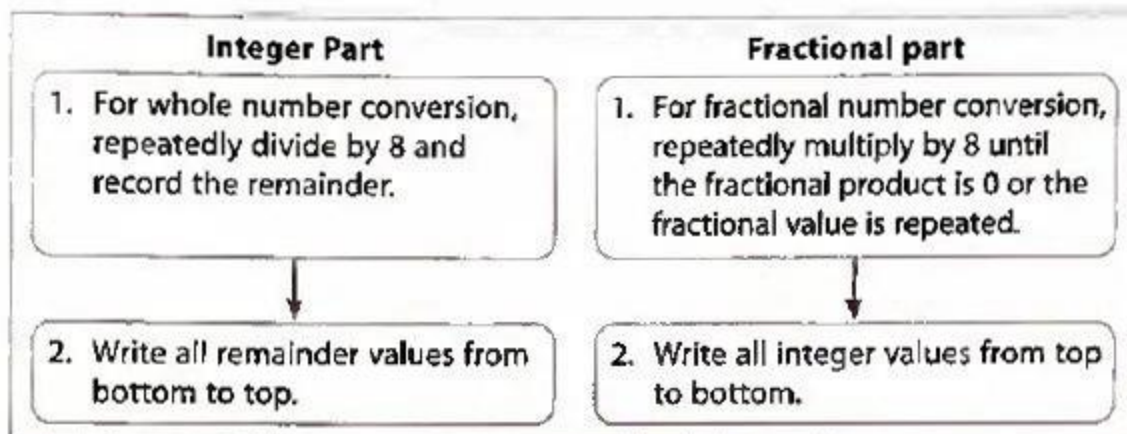
Reading the integers from top to bottom $(0.15)_{10} = (0.001001)_2$

Final result $(105.15)_{10} = (1101001.001001)_2$

Note: If the fractional part does not become 0 in successive multiplication or start repeating, then stop further calculation.

Decimal to Octal Conversion

Method to convert a Decimal number into Octal number:



Step 1: Divide the given decimal number by 8 and record its quotient Q_1 and remainder R_1 .

Step 2: Again divide the new quotient obtained from Step 1 by 8 and record the new quotient Q_2 and remainder R_2 .

Step 3: Repeat Step 1 and Step 2 until quotient Q_n becomes 0.

Step 4: Write all remainder values R_n moving from bottom to top, i.e., moving from most significant bit (MSB) to least significant bit (LSB).

Example 5:

Convert the Decimal number 125 into its Octal equivalent.

8	125	Remainder	LSB
8	15	5	↑
8	1	7	
	0	1	
			MSB

Learning Tip: For converting decimal number into octal number, we divide decimal number by 8.

Answer: $(125)_{10} = (175)_8$

Converting Decimal fraction into Octal fraction

Method to convert a Decimal fraction into Octal fraction:

1. Multiply the given fraction by 8.
2. Keep the integer in the product as it is and multiply the new fraction in the product by 8.
3. Continue the process and read the integers in the products from top to bottom.

Example 6:

Convert 0.21875 into octal fraction.

Given fraction 0.21875

	(fraction)	(integer)
$0.21875 \times 8 = 1.75000$	.75000	1 (LSB)
		
$0.75000 \times 8 = 6.00000$	.00000	6 (MSB)

Therefore, $(0.21875)_{10} = (0.16)_8$

Decimal to Hexadecimal Conversion

Method to convert a Decimal number into its Hexadecimal equivalent:

- Step 1:** Divide the given decimal number by 16 and record its quotient Q_1 and remainder R_1 .
- Step 2:** Again divide the new quotient obtained from Step 1 by 16 and record the new quotient Q_2 and remainder R_2 .
- Step 3:** Repeat Step 1 and Step 2 until quotient Q_n becomes 0.
- Step 4:** Write all remainder values R_n moving from bottom to top, i.e., moving from most significant bit (MSB) to least significant bit (LSB).

Example 7:

Convert the Decimal number 300 into its Hexadecimal equivalent.

16	300	Remainder	LSB
16	18	12-(C)	
16	1	2	
		1	
			MSB

Learning Tip: For converting decimal number into hexadecimal number, we divide decimal number by 16.

Answer: $(12C)_{16}$

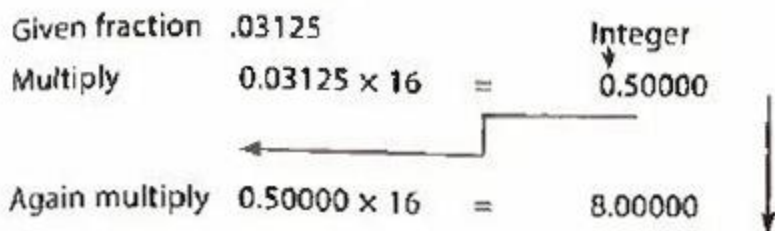
Converting Decimal fraction into Hexadecimal fraction

Method to convert a Decimal fraction into Hexadecimal fraction:

1. Multiply the given fraction by 16.
2. Keep the integer in the product as it is and multiply the new fraction in the product by 16.
3. Continue the process and read the integers in the products from top to bottom.

Example 8:

Convert $(0.03125)_{10}$ into hexadecimal fraction.



Stop multiplying fractional part by 16 as the fractional part is now 0.

Move from top to bottom and write all integer values. The result is:

$$(0.03125)_{10} = (0.08)_{16}$$

2.3.2 Other Base to Decimal Number System

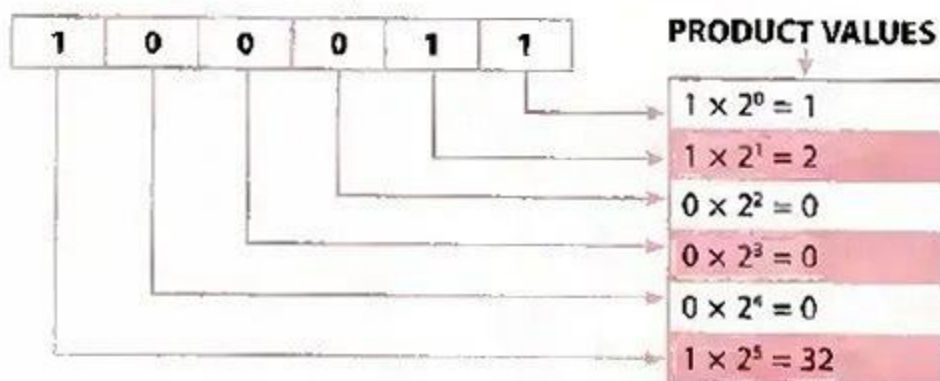
Method to convert Binary into Decimal:

1. Find the weight of each bit position, i.e., power of 2.
2. Start at the right-most bit.
3. Take that bit and multiply by 2^n where n is the current position, i.e., 2^0 and increasing by 1 each time to the left.
4. Sum each term of product until all bits have been used.

Binary to Decimal

For example,

Q. Convert Binary Number $(100011)_2$ into Decimal Number.



Ans. By adding all these product values, we get:

$$= 1 + 2 + 0 + 0 + 0 + 32 = 35$$

$$\text{Hence, } (100011)_2 = (35)_{10}$$

- Convert the Binary number 101011 into its Decimal equivalent.

$$1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$32 + 0 + 8 + 0 + 2 + 1 = (43)_{10}$$

- Convert the Binary number 1001 into its Decimal equivalent.

$$1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$8 + 0 + 0 + 1 = (9)_{10}$$

Learning Tip: For converting other base number into decimal number equivalent, each digit is multiplied by its respective positional weight and then added to get the desired result.

Binary fraction to Decimal Conversion

For example,

Convert $(11011.1101)_2$ into decimal.

$$\begin{array}{ccccccccc} 2^4 & 2^3 & 2^2 & 2^1 & 2^0 & & 2^{-1} & 2^{-2} & 2^{-3} & 2^{-4} \\ 1 & 1 & 0 & 1 & 1 & , & 1 & 1 & 0 & 1 \\ = (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) + (1 \times 2^{-2}) + (0 \times 2^{-3}) + (1 \times 2^{-4}) \\ = 16 + 8 + 0 + 2 + 1 + 0.5 + 0 + 0.25 + 0.0625 \\ = (27.8125)_{10} \end{array}$$

Octal to Decimal Conversion

Method to convert Octal into Decimal:

1. Find the weight for each bit position, i.e., powers of 8.
2. Start at the right-most bit.
3. Take that bit and multiply by 8^n where n is the current position, i.e., 8^0 , and increasing by 1 each time to the left.
4. Total each of the product terms until all bits have been used.

For example,

Convert the Octal number 321 into its Decimal equivalent.

$$3 \times 8^2 + 2 \times 8^1 + 1 \times 8^0$$

$$192 + 16 + 1 = (209)_{10}$$

Octal fraction to Decimal Conversion

For example,

Convert $(23.25)_8$ into decimal.

$$\begin{array}{cccc} 8^1 & 8^0 & & 8^{-1} & 8^{-2} \\ 2 & 3 & , & 2 & 5 \\ = (2 \times 8^1) + (3 \times 8^0) + (2 \times 8^{-1}) + (5 \times 8^{-2}) \\ = 16 + 3 + 0.25 + 0.078125 \\ = (19.328125)_{10} \end{array}$$

Hexadecimal to Decimal Conversion

Method to convert Hexadecimal into Decimal:

1. Find the weight for each bit position, i.e., powers of 16.
2. Start at the right-most bit.
3. Take that bit and multiply by 16^n where n is the current position beginning, i.e., 16^0 , and increasing by 1 each time to the left.
4. Total each of the product terms until all bits have been used.

For example,

Convert the Hexadecimal number AB into its Decimal equivalent.

$$\begin{aligned} &= A \times 16^1 + B \times 16^0 \\ &= 10 \times 16^1 + 11 \times 16^0 \\ &= 160 + 11 = (171)_{10} \end{aligned}$$

Hexadecimal fraction to Decimal Conversion

Example 9:

Convert $(1E.8C)_{16}$ into decimal.

$$\begin{array}{cccc} 16^1 & 16^0 & 16^{-1} & 16^{-2} \\ 1 & E & 8 & C \\ = (1 \times 16^1) + (14 \times 16^0) + (8 \times 16^{-1}) + (12 \times 16^{-2}) \\ = 16 + 14 + 0.5 + 0.046875 \\ = (30.546875)_{10} \end{array}$$

2.3.3 One Base to Other Base System

Binary to Hexadecimal Conversion

Each alphanumeric digit is represented as a group of four binary digits in hexadecimal system.

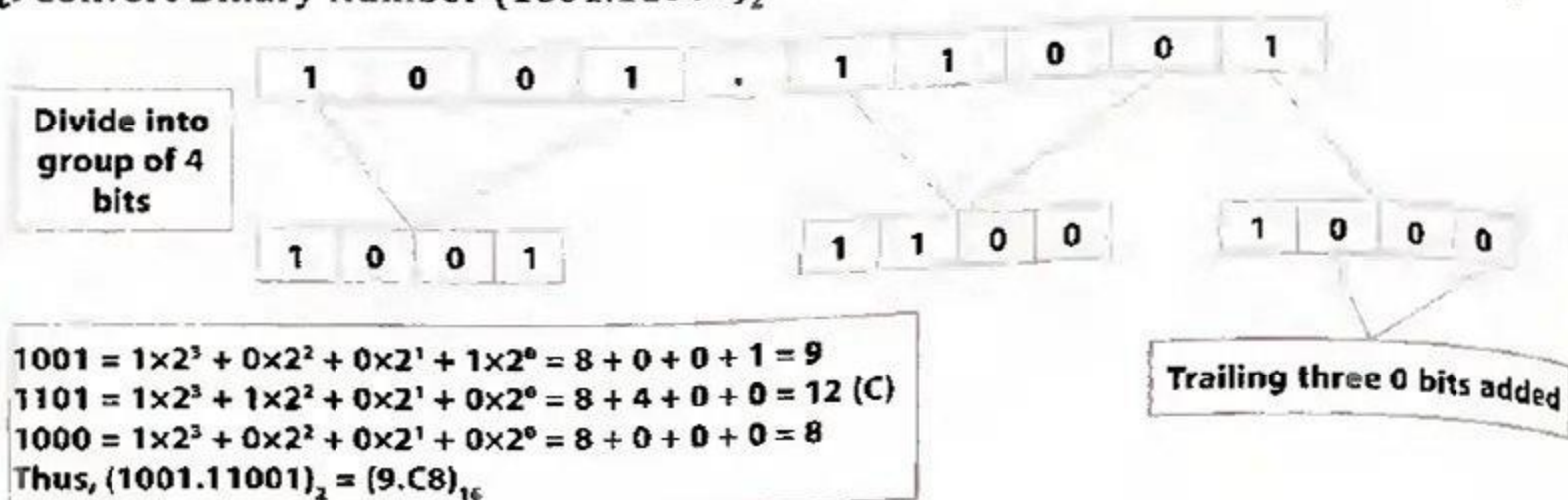
Binary Number	Hexadecimal Number
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Method to convert a Binary number into its Hexadecimal equivalent

We take a binary number in groups of four and use the appropriate hexadecimal digit in its place. We begin at the right-most 4 bits. If we are not able to form a group of four, insert 0s to the left until we get all groups of 4 bits each. Write the hexadecimal equivalent of each group. Repeat the steps until all groups have been converted.

For example,

Q. Convert Binary Number $(1001.11001)_2$ into its equivalent Hexadecimal Number.



➤ Convert the Binary number 1000101 into its Hexadecimal equivalent.

0100 0101 Note that we needed to insert a 0 to the left of 100.
 4 5

Ans. $(45)_{16}$

In case of a fractional binary number, form groups of four bits on each side of decimal point. Then replace each group by its corresponding hexadecimal number.

For example,

➤ Convert $(11100.1010)_2$ into hexadecimal equivalent.

0001 1100 1010
 1 C A

Ans. $(1C.A)_{16}$

Hexadecimal to Binary Conversion

Method to convert a Hexadecimal number into its Binary equivalent

Convert each digit of Hexadecimal number into its binary equivalent and write them in 4 bits. Then combine each 4-bit binary number and that is the resulting answer.

Example 10:

Convert the Hexadecimal number $(10AF)_{16}$ into its Binary equivalent.

1 0 A F
 0001 | 0000 | 1010 | 1111

Ans. $(0001000010101111)_2$

Example 11:

Convert the Hexadecimal number $(A2F)_{16}$ into its Binary equivalent.

A 2 F
 1010 | 0010 | 1111

Ans. $(1010 0010 1111)_2$

Q. Convert the Hexadecimal Number (FACE.32)₁₆ into Binary Number.

Ans.

F	→	1	1	1	1
A	→	1	0	1	0
C	→	1	1	0	0
E	→	1	1	1	0
3	→	0	0	1	1
2	→	0	0	1	0

Hence,

$$(FACE.32)_{16} = (1111101011001110.00110010)_2$$

Binary to Octal and Octal to Binary Conversion

The octal system is a power of 8 and can be made from binary numbers by grouping consecutive binary digits into groups of three, i.e., (2^3) . To convert Binary into Octal, we can take the bits into groups of 3 and represent each group as an octal digit. Insert 0s to the left until we get all groups of 3 bits each. The steps are the same for the binary to hexadecimal conversions except we are dealing with the octal base now.

* $8 = 2^3$

* Each group of 3 bits represents an octal digit

Example:

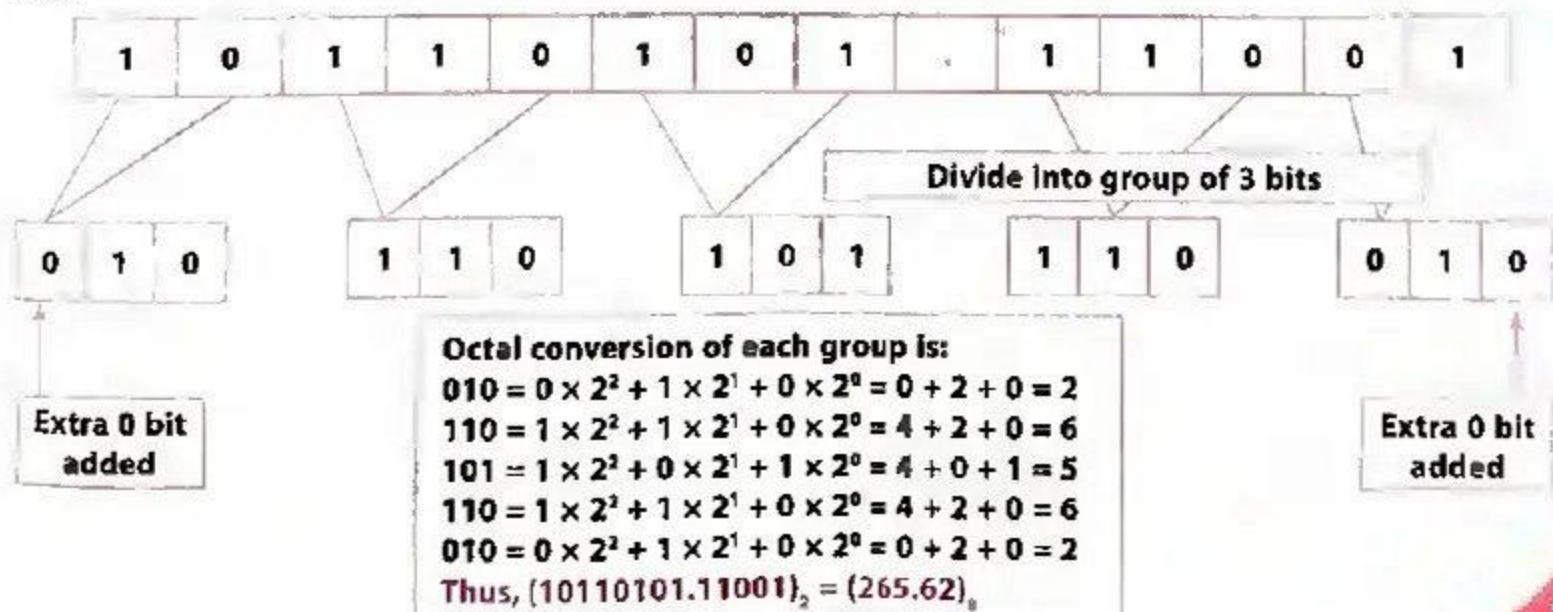
$$\begin{array}{ccc} \text{Assume Zeros} \\ (010 \ 110 \ . \ 010)_2 \\ \downarrow \quad \downarrow \quad \downarrow \\ (2 \quad 6 \quad . \quad 2)_8 \end{array}$$

Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

To convert from octal to binary, we simply represent each octal digit in its three-bit binary form as shown in the above figure.

Q. Convert Binary Number (10110101.11001)₂ into Octal Number.

Ans.



For example,

- Convert the Octal number $(742)_8$ into its Binary equivalent.

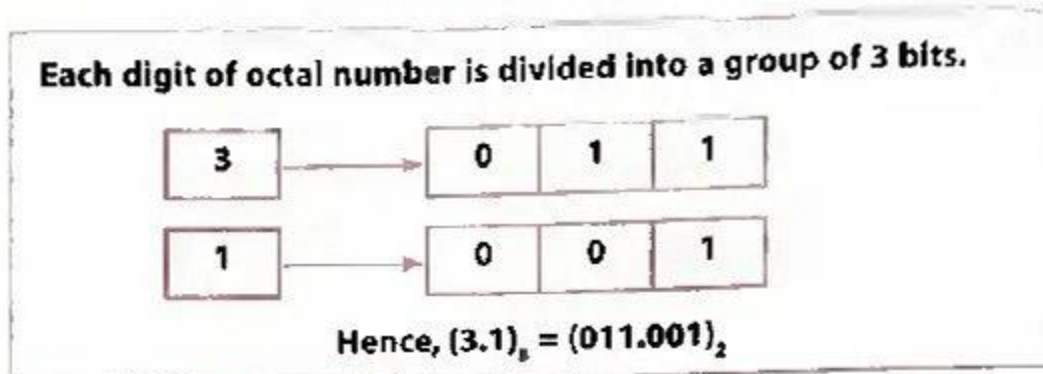
7 | 4 | 2

111 | 100 | 010

Answer: $(111100010)_2$

Q. Convert Octal Number $(3.1)_8$ into its equivalent Binary Number.

Ans.



Hexadecimal to Octal and Octal to Hexadecimal Conversion

To convert Hexadecimal into Octal, convert each digit of Hexadecimal number into its binary equivalent and write them in 4 bits. Then, combine each 3-bit binary number and that is converted into octal.

For example,

- Convert the Hexadecimal number $(A42)_{16}$ into its Octal equivalent.

A | 4 | 2

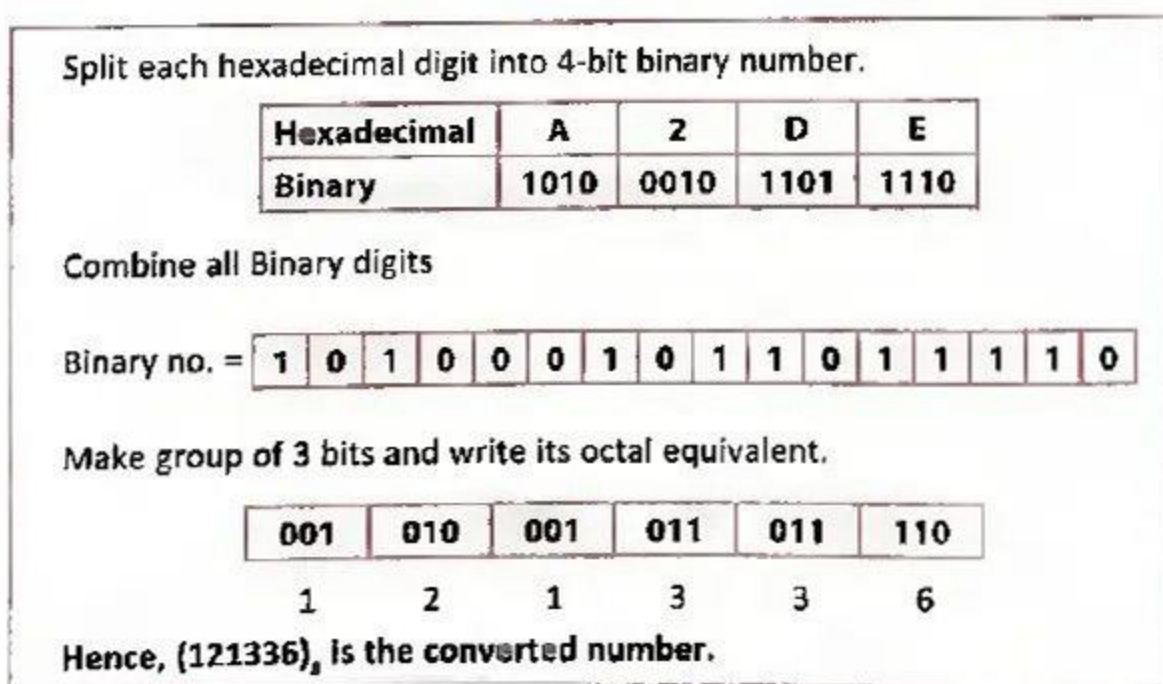
1010 | 0100 | 0010

101 | 001 | 000 | 010

Answer: $(5102)_8$

Q. Convert Hexadecimal number $(A2DE)_{16}$ into its equivalent Octal number.

Ans.



To convert Octal into Hexadecimal, convert each digit of Octal number into its binary equivalent and write them in 3 bits. Then combine each 4-bit binary number and that is converted into hexadecimal.

For example,

➤ Convert the Octal number $(762)_8$ into its hexadecimal equivalent.

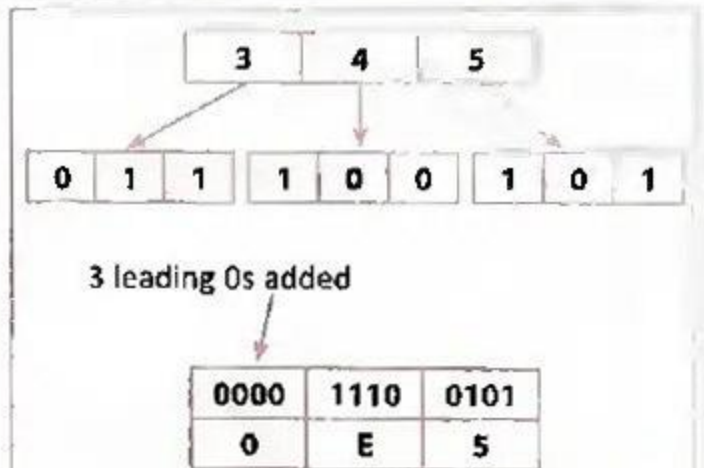
7		6		2
101		110		010
0001		0111		0010
1		7		2

Answer: $(172)_{16}$

Q. Convert Octal Number $(345)_8$ into its equivalent Hexadecimal number.

Ans.

- Octal Number = 345
- Divide each digit into 3 bits.
- Combine all bits:
011100101
- Make group of 4 bits and convert into hexadecimal equivalent.
- The converted number is:
 $(E5)_{16}$



The following table summarizes the number representation in decimal, binary, octal and hexadecimal number system:

Table 2.2: Number representation in decimal, binary, octal and hexadecimal number system

Decimal	Binary	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

2.4 INTERNAL STORAGE ENCODING OF CHARACTERS

The data in computers is stored in a binary form. The data can be numeric data or non-numeric data, which can be alphabets or any other special characters. These non-numeric data are said to be alphanumeric data. Therefore, when a character, letter or symbol is pressed on the keyboard, it is internally mapped to a unique code which is further converted to binary. The way of converting data from one format to another is called encoding scheme. In this process, we can assign alphanumeric code to specific characters and convert those codes in binary language. These binary numbers later can be converted back to original characters based on their values. There are some predefined encoding schemes which are used to represent the numeric and non-numeric characters.

The various encoding schemes for data representation are:

- ASCII (American Standard Code for Information Interchange)
- ISCII (Indian Standard Code for Information Interchange)
- UNICODE (Universal Coding Standard)

2.4.1 ASCII (American Standard Code for Information Interchange)

ASCII is the most widely-used alphanumeric code which is used in computers to translate text (letters, numbers, and symbols) into a form that can be sent to and understood by other computers and devices such as modems and printers. It is developed by the American National Standards Institute (ANSI). The standard ASCII character set uses just 7 bits for each character and so it has $2^7 = 128$ possible code groups. It represents all of the standard keyboard characters as well as control functions such as Return & Linefeed functions. The characters encoded are numbers 0 to 9, lowercase letters a to z, uppercase letters A to Z, basic punctuation symbols, control codes that originated with Teletype machines, and a space.

Table 2.3: Standard or Lower ASCII Characters and Codes

Char	Dec	Binary	Char	Dec	Binary	Char	Dec
1	033	00100001	A	065	01000001	a	097
"	034	00100010	B	066	01000010	b	098
#	035	00100011	C	067	01000011	c	099
\$	036	00100100	D	068	01000100	d	100
%	037	00100101	E	069	01000101	e	101
&	038	00100110	F	070	01000110	f	102
'	039	00100111	G	071	01000111	g	103
(	040	00101000	H	072	01001000	h	104
)	041	00101001	I	073	01001001	i	105
*	042	00101010	J	074	01001010	j	106
+	043	00101011	K	075	01001011	k	107
,	044	00101100	L	076	01001100	l	108
-	045	00101101	M	077	01001101	m	109
.	046	00101110	N	078	01001110	n	110
/	047	00101111	O	079	01001111	o	111
0	048	00110000	P	080	01010000	p	112
1	049	00110001	Q	081	01010001	q	113

2	050	00110010	R	082	01010010	r	114
3	051	00110011	S	083	01010011	s	115
4	052	00110100	T	084	01010100	t	116
5	053	00110101	U	085	01010101	u	117
6	054	00110110	V	086	01010110	v	118
7	055	00110111	W	087	01010111	w	119
8	056	00111000	X	088	01011000	x	120
9	057	00111001	Y	089	01011001	y	121
:	058	00111010	Z	090	01011010	z	122
:	059	00111011	[	091	01011011	[	123
<	060	00111100	\	092	01011100		124
=	061	00111101	]	093	01011101	]	125
>	062	00111110	^	094	01011110	~	126
?	063	00111111	_	095	01011111	~	127
@	064	01000000	`	096	01100000		

Q. Write the ASCII-7 coding for the word BYTE in both binary and hexadecimal notations. How many bytes are required to store this word using this coding?

Ans. In ASCII-7 code, the word BYTE can be written as:

B = 1000010 in binary and 42 in hexadecimal

Y = 1011001 in binary and 59 in hexadecimal

T = 1010100 in binary and 54 in hexadecimal

E = 1000101 in binary and 45 in hexadecimal

Hence, ASCII-7 coding for the word BYTE will be 1000010101100110101001000101 in binary and 42595445 in hexadecimal.

Since each character in ASCII-7 requires one byte for its representation and there are 4 characters in the word BYTE, so 4 bytes will be required to store this word using this coding.

Q. Encode the word DATA and convert the encoded value into binary values which can be understood by a computer.

Ans. In ASCII, value of D is 68 and its equivalent 7-bit binary code = 1000100

- ASCII value of A is 65 and its equivalent 7-bit binary code = 1000001
- ASCII value of T is 84 and its equivalent 7-bit binary code = 1010100
- ASCII value of A is 65 and its equivalent 7-bit binary code = 1000001

Replace each alphabet in DATA with its ASCII code value to get its equivalent ASCII code and with 7-bit binary code to get its equivalent binary number as shown in Table 2.4.

Table 2.4: ASCII and equivalent Binary values for word DATA

	D	A	T	A
ASCII Code	68	65	84	65
Binary Code	1000100	1000001	1010100	1000001

2.4.2 ISCII (Indian Standard Code for Information Interchange)

In 1991, the Bureau of Indian Standards adopted the ISCII. It is an 8-bit code which allows English and Indian scripts' alphabets to be used simultaneously. Characters coded in ISCII need 8 bits for each character. To use Indian languages on computers, ISCII codes are used. It is an 8-bit code capable of representing 256 characters.

ISCII code retains all ASCII characters and offers coding for Indian scripts also. Thus, it is also known as Indian Scripts Code for Information Interchange. There are 15 officially recognized languages in India: Hindi, Marathi, Sanskrit, Punjabi, Gujarati, Oriya, Bengali, Assamese, Telugu, Kannada, Malayalam, Tamil, Urdu, Sindhi and Kashmiri. Out of these, Urdu, Sindhi and Kashmiri are primarily written in Perso-Arabic scripts, besides in Devanagari.

	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17
240																
250																
300																
320																
340																
360	EXT															

Gujarati

	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17
240																
250																
300																
320																
340																
360	EXT															

Devanagari

Fig. 2.4: ISCII Code chart for Gujarati and Devanagari characters

Characters coded in ISCII need 8 bits for each character. This standard not only applies to the Devanagari script but also to Gujarati, Bengali, Gurmukhi, Assamese, Telugu, Kannada, Malayalam and Tamil scripts.

2.4.3 UNICODE (for Multilingual Computing)

Unicode is a new universal coding standard adopted by all new platforms. It is promoted by Unicode Consortium which is a non-profit organization. Unicode provides a unique number for every character irrespective of the platform, program and the language. It is a character coding system designed to support the worldwide interchange, processing and display of the written texts of the diverse languages. A unique number for every character, no matter what the platform, no matter what the program, no matter what the language.

Significance of UNICODE

- Unicode enables a single software product or a single website to be designed for multiple platforms, languages and countries (no need for re-engineering) which can lead to a significant reduction in cost over the use of legacy character sets.
- Unicode data can be used through many different systems without data corruption.
- Unicode represents a single encoding scheme for all languages and characters.
- Unicode is a common point in the conversion between other character encoding schemes. Since it is a superset of all of the other common character encoding systems, you can convert from one encoding scheme to Unicode, and then from Unicode to the other encoding scheme.
- Unicode is the preferred encoding scheme used by XML-based tools and applications.

CTM: Unicode is a way to include all the different schemes into one universal text-encoding.

UTF 8, UTF 16, UTF 32

UTF stands for Unicode Transformation Format. UTF was developed so that users have a standardized means of encoding the characters with the minimal amount of space.

Unicode and Code Points

Unicode character set mapped each character in the world to a unique number. This ensured that there are no collisions between alphabets of different languages. These numbers are platform-independent. These unique numbers are called code points in the Unicode terminology.

UTF-8, UTF-16 and UTF-32 are the only few established standards for encoding.

UTF-8 Encoding

UTF-8 encoding is a variable-sized encoding scheme to represent Unicode code points in memory. Variable-sized encoding means the code points are represented using 1, 2, 3 or 4 bytes depending on their size.

UTF-16 Encoding

UTF-16 encoding is a variable-byte encoding scheme which uses either 2 bytes or 4 bytes to represent Unicode code points. Most of the characters for all modern languages are represented using 2 bytes.

UTF-32 Encoding is a fixed-byte encoding scheme and it uses 4 bytes to represent all code points.

UTF-8, UTF-16 and UTF-32 are the different ways to represent them in byte format. Both UTF-8 and UTF-16 are variable length encoding, where number of bytes used depends upon Unicode code points. On the other hand, UTF-32 is a fixed width encoding, where each code point takes 4 bytes. Unicode contains code points for almost all representable graphic symbols in the world and it supports all major languages, *e.g.*, English, Japanese, Mandarin or Devanagari.

The main advantage of UTF-8 is that it is backward compatible with ASCII. The ASCII character set is fixed width and only uses one byte. When encoding a file that uses only ASCII characters with UTF-8, the resulting file would be identical to a file encoded with ASCII. This is not possible when using UTF-16 as each character would be two bytes long.

2.5 BOOLEAN LOGIC

Have you ever wondered how a computer can do something like balancing a chequebook, or playing chess or spell-checking a document? These are things that, just a few decades ago, only humans could do. Now, computers do them with much ease. How can a “chip” made up of silicon and wires do something that seems like it requires human thought?

If you want to know the answer to this question, the first thing you need to understand is something called Boolean logic. Boolean logic was originally developed by **George Boole** in the mid-1800s, who figured out a way to put formal logic into mathematical form. Simply put, Boolean logic is a very easy way to figure out the truth of an expression using the simple concept of true or false.

Consider the following examples:

- Should I go to a restaurant or not?
- Should I purchase a new mobile phone or not?
- Should I opt for Computer Science or not?

All these statements can have only one answer, either in the form of yes or no. Thus, a binary decision always results in either yes/no or true/false. Likewise, a switch has only two states—open and closed. An open switch represents binary 0 and a closed switch represents binary 1.

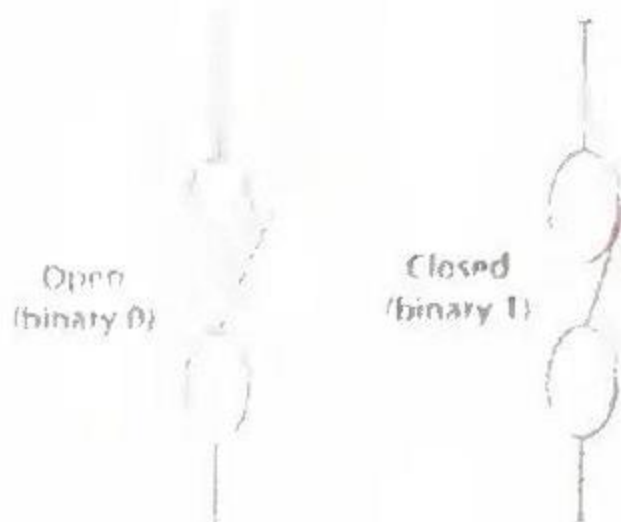


Fig. 2.5: Showing Open and Closed Switch

The digit 1 or letter 'T' is used to represent true value and the digit 0 or letter 'F' represent false value. A statement which evaluates either true or false is known as Proposition (or statement).

CTM: 1 and 0 are two binary-valued quantities that represent true or false state.

Boolean Variable

A Boolean variable is also known as **binary variable** or **logical variable** that takes values from Boolean algebra. A Boolean variable can take only one binary-valued quantity out of the two possible values, i.e., yes/no, 1/0, true/false. One bit represents one Boolean variable. In electrical circuit, if the signal passes, it represents 1, else it is represented through 0.

Boolean Constant

The values which are stored in binary variables are known as **Boolean Constants**. Therefore, the values **true/false, yes/no** or **1/0** are Boolean constants.

Boolean Statement: A statement is said to be a Boolean/Logical Statement if it has a definite value which is either false or true.

Examples:

Logical Statement	Non-logical Statement
Have you finished your homework?	What are your plans for tomorrow?
Is the house locked?	Who is the President of India?
Is New Delhi the capital of India?	Where is your house located?

Boolean Variable: A variable which holds false/true value is known as Boolean variable (X, Y, Z, etc.)

Boolean Expression: A meaningful combination of Boolean operators (AND/OR/NOT), Boolean operand/variable (X, Y, Z, etc.) and Boolean constant (0 or 1) is known as Boolean Expression (Logical Expression). For example,

- (i) $X \leftarrow Y \cdot Z$
- (ii) $A \cdot (B + C) + B \cdot C$
- (iii) $D \text{ OR } V \text{ AND NOT } Z$

2.0.1 AND OPERATION

operations used in Boolean algebra are known as Boolean/logical operations. Most of the operations and their notations are shown below.

EXAMPLES

$$x \cdot y$$

$$x \wedge y$$

$$x \& y$$

2.0.1 AND Operator

AND operator is a binary operator that operates on two variables and the operation performed by AND operator is known as logical multiplication. The symbol used for logical multiplication is dot ($\cdot$) operator.

Consider the following values for two variables in a given Table 1. Here, when there is logical multiplication using AND operator, the result will be as shown in Table 2.

Table 1		Table 2		
A	B	A	B	OUTPUT A.B
0	0	0	0	0
0	1	0	1	0
1	0	1	0	0
1	1	1	1	1

The AND operation will result in true value (1) when both inputs are 1 (true/high), and for all other values it results in 0 (false/low).

2.0.2 OR Operator

The OR operator is a binary operator that operates on two variables and the operation performed by OR operator is known as logical addition. The symbol used for logical addition is plus ($+$) operator.

Consider the following values for two variables A and B as shown in Table 1. When there is logical addition using OR operator, the result will be as shown in Table 2.

Table 1		Table 2		
A	B	A	B	OUTPUT A+B
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	1	1	1

The OR operation results in true value (1) when either of the inputs is 1 (true) or both the inputs are 1 (true/high), and for all other values of inputs it results in 0 (false/low).

2.0.3 NOT Operator

The NOT operator is a unary operator that operates on one variable and the operation performed by NOT operator is known as negation or complementation.

Consider Tables 1 and 2 with values 0 and 1 for variable A.

Table 1

A
0
1

Table 2

A	OUTPUT A'
0	1
1	0

It means that the logical statements A and A' are opposite to each other. If the value of a variable A is 0, its complement would be 1, and if the value of the variable A is 1, its complement would be 0.

2.7 TRUTH TABLE

A truth table is a representation of a Boolean function or expression containing all possible combinations of input values and their result in a tabular format. If the result of a logical expression or logical statement is always true or 1, it is known as **Tautology**. If the result is always false or 0, it is known as **Fallacy**.

The number of rows in a truth table is calculated as 2^n where n is the total number of Boolean variables. Therefore, if there are two variables A and B, then the possible input values will be $2^2 = 4$ combinations. It can be represented as follows:

A	B	$A \vee B$	$A \wedge B$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1

If there are three variables A, B and C, then the possible input values will be $2^3 = 8$ combinations. It can be represented as follows:

A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Rules for writing Truth Table

1. First check the number of literals or variables in a Boolean expression.
2. Draw columns for each unique variable. For example, if there is a Boolean expression as $A+B.C$, then there will be three columns, one for each variable.
3. Draw columns for each logical expression. In a Boolean expression $A+B.C$, there are two logical operations, viz. AND and OR operations.
4. Draw rows which can be calculated as 2^n where n is the total number of variables. In the example given above, there are three variables A, B and C. Hence, total number of rows will be $2^3 = 8$.
5. Write 0's in the first half for the number of rows in the first column and in the second half, for number of rows, write 1's.

6. Similarly, write 0's in the first half for the number of rows which have 0's in the first column and 1's in the same number of rows. Repeat this pattern till 0's and 1's have been written for all 0's and 1's of the first column.
7. This pattern will continue till you reach the last column with the pattern 0's and 1's alternatively.
8. Calculate the result for each logical operation by considering the values for input variables written in each row.

Let us discuss some questions on creating truth table for Boolean expressions.

Q. Write a truth table for the Boolean expression: $F = A.B' + C$ by writing each step.

Ans. Follow the given steps for writing the truth table:

1. In the given expression, there are three variables A, B and C.
2. Draw three columns for each possible input variable combination and three columns for each logical operations NOT, AND and OR and $2^3 = 8$ rows such as:

A	B	C	B'	A.B'	A.B' + C
---	---	---	----	------	----------

3. Fill the first half for number of rows with 0's and second half number of rows with 1's in the first column for variable A. Follow rule nos. 6 and 7 for all other columns for variables B and C.

A	B	C	B'	A.B'	A.B' + C
0	0	0	1	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	1	0	0	1

4. Perform logical operations B', A.B' and A.B' + C by considering the values 0's and 1's written in each row for each input variable combination and filling the rows for these columns with 0's and 1's. The resultant table will be a truth table.

A	B	C	B'	A.B'	A.B' + C
0	0	0	1	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	1	0	0	1

CTM: Number of columns = Number of variables + Number of logical expressions

Number of rows = 2^n where n is the total number

Q. Write a truth table for the given Boolean Expression: $F = A + B.C' + A'.C'$

Ans. $F = A + B.C' + A'.C'$ [Given]

A	B	C	C'	A'	B.C'	A'.C'	A+B.C'	A+B.C'+A'.C'
0	0	0	1	1	0	1	0	1
0	0	1	0	1	0	0	0	0
0	1	0	1	1	1	1	1	1
0	1	1	0	1	0	0	0	0
1	0	0	1	0	0	0	1	1
1	0	1	0	0	0	0	1	1
1	1	0	1	0	1	0	1	1
1	1	1	0	0	0	0	1	1

➤ Precedence of Boolean Operators

1. NOT (First Precedence)
2. AND (Second Precedence)
3. OR (Third Precedence)



The NOT operator has the highest priority and OR has the lowest priority. For evaluating the Boolean expression, the following rules should be followed:

➤ Rules for Evaluating Boolean Expression

1. Evaluate the Boolean expression from left to right.
2. Evaluate the expression in parentheses first.
3. Perform all NOT expressions.
4. Perform all AND expressions.
5. Perform all OR expressions.

Precedence of operators can also be illustrated with the help of a truth table. Consider a Boolean expression $X.Y' + Z$

X	Y	Z	Y'	X.Y'	X.Y'+Z
0	0	0	1	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	1	0	0	1

In this truth table, NOT operation is performed first followed by AND operation and then OR operation is performed.

Example 12: Prove the following with the help of a truth table:

$$x.(y+z) = (x.y) + (x.z)$$

x	y	z	y+z	x.(y+z)	x.y	x.z	(x.y) + (x.z)
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

CTM: The order of precedence for logical operators is NOT, AND, OR.

2.8 LOGIC CIRCUIT

A logic circuit is the one which takes one or more inputs and generates an output. Logic Gates use the binary operators AND, OR and NOT. There are three fundamental Logic Gates which are as follows:

1. NOT Gate
2. AND Gate
3. OR Gate

NOT Gate

A NOT gate, sometimes called a logical *inverter gate* to differentiate it from other types of electronic inverter devices, has only one input. It reverses the logic state. The output state is a negation or complement of an input signal.

AND Gate

The AND gate is so named because if 0 is called "false" and 1 is called "true," the gate acts in the same way as the logical "AND" operator. The output is "true" when both the inputs are "true." Otherwise, the output is "false." AND Gate works on two or more inputs which result in a single output.

OR Gate

The OR gate gets its name from the fact that it behaves like logical inclusive "OR." The output is "true" if either or both of the inputs are "true." If both inputs are "false," then the output is "false." OR Gate works on two or more inputs which result in a single output.

Some other important Logic Gates are:

NOR Gate

The NOR gate is a combination OR gate followed by an inverter NOT gate. Its output is "true" if both the inputs are "false." Otherwise, the output is "false."

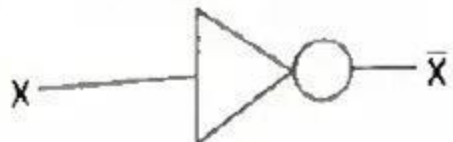
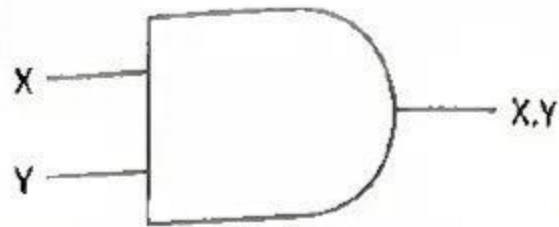
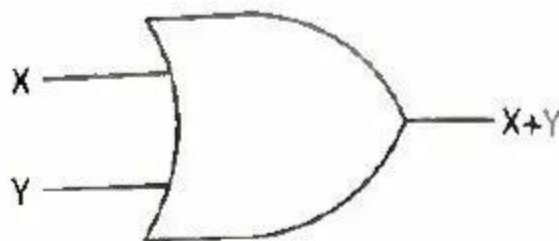
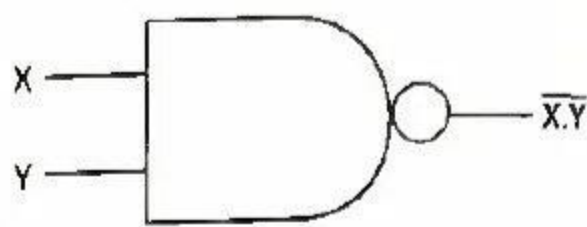
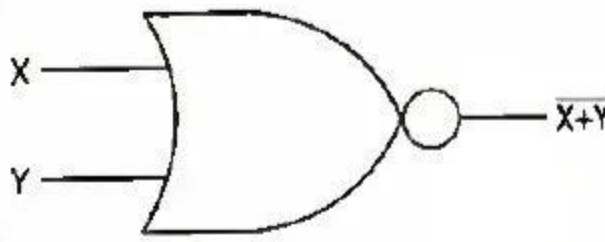
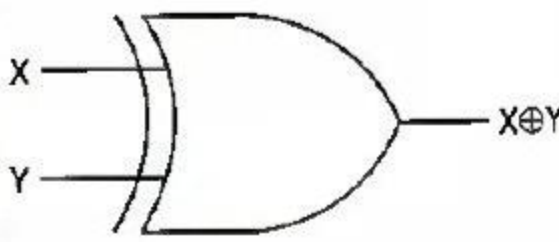
NAND Gate

The NAND gate operates as an AND gate followed by a NOT gate. It acts in the manner of logical operator "and" followed by negation. The output is "false" if both the inputs are "true." Otherwise, the output is "true".

XOR Gate

The XOR (exclusive-OR) gate acts in the same way as the logical "either/or." The output is "true" if either, but not both, of the inputs is "true." The output is "false" if both the inputs are "false" or if both the inputs are "true." Another way of looking at this circuit is to observe that the output is 1 if the inputs are different, but 0 if the inputs are the same.

The following table shows the truth table and logic gate of various operators.

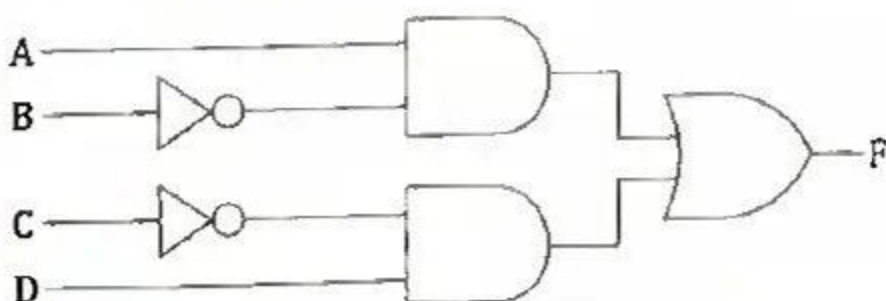
Operator	Truth Table	Logic Gate																				
NOT	<table><tr><th>X</th><th>X'</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	X	X'	0	1	1	0															
X	X'																					
0	1																					
1	0																					
AND	<table><tr><th>X</th><th>Y</th><th>X.Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	X	Y	X.Y	0	0	0	0	1	0	1	0	0	1	1	1						
X	Y	X.Y																				
0	0	0																				
0	1	0																				
1	0	0																				
1	1	1																				
OR	<table><tr><th>X</th><th>Y</th><th>X+Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	X	Y	X+Y	0	0	0	0	1	1	1	0	1	1	1	1						
X	Y	X+Y																				
0	0	0																				
0	1	1																				
1	0	1																				
1	1	1																				
NAND	<table><tr><th>X</th><th>Y</th><th>X.Y</th><th>(X.Y)'</th></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	X	Y	X.Y	(X.Y)'	0	0	0	1	0	1	0	1	1	0	0	1	1	1	1	0	
X	Y	X.Y	(X.Y)'																			
0	0	0	1																			
0	1	0	1																			
1	0	0	1																			
1	1	1	0																			
NOR	<table><tr><th>X</th><th>Y</th><th>X+Y</th><th>(X+Y)'</th></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	X	Y	X+Y	(X+Y)'	0	0	0	1	0	1	1	0	1	0	1	0	1	1	1	0	
X	Y	X+Y	(X+Y)'																			
0	0	0	1																			
0	1	1	0																			
1	0	1	0																			
1	1	1	0																			
XOR	<table><tr><th>X</th><th>Y</th><th>X⊕Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	X	Y	X⊕Y	0	0	0	0	1	1	1	0	1	1	1	0						
X	Y	X⊕Y																				
0	0	0																				
0	1	1																				
1	0	1																				
1	1	0																				

CTM: The NAND and NOR gates are called universal gates because any digital circuits can be implemented using these gates.

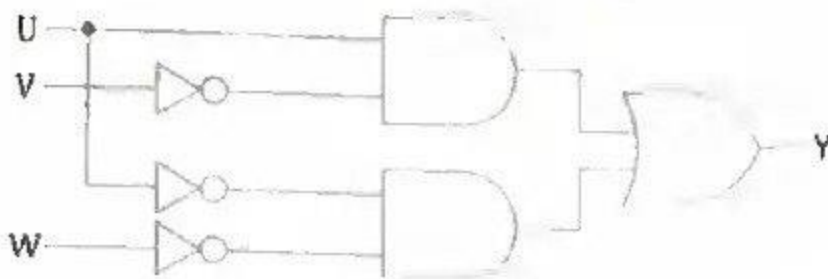
Q. Draw a circuit diagram corresponding to the following Boolean expression:

$$F = A.B' + C'.D$$

Ans.

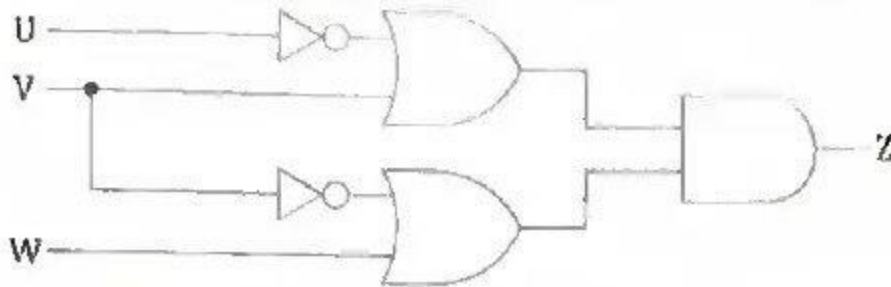


Q. Write equivalent Boolean expression for the following Logic Circuit:



Ans. $Y = (U.V') + (U'.W')$

Q. Write equivalent Boolean expression for the following Logic Circuit:



Ans. $Z = (U'+V).(V'+W)$

Duality Principle in Boolean Algebra

This law explains that replacing the variables doesn't change the value of the Boolean function. The dual of the expressions can be achieved by:

1. Replacing each AND (.) sign with an OR (+) sign.
2. Replacing each OR (+) sign with an AND (.) sign.
3. Replacing each variable, such as replacing 1 with 0 and replacing 0 with 1.

If the operators and variables of an equation or function produce no change in the output of the equation, though they are interchanged, it is called "Duals".

Q. Find the dual of the following expressions:

(i) $A.B+A.C$

Ans. $(A+B).(A+C).$

(ii) $x\bar{y} + \bar{x}z$

Ans. $(x + \bar{y}) \cdot (\bar{x} + z)$

Some Rules/Laws of Boolean Algebra

	Law Name	AND	OR
1	Property of 0	$0.A = 0$	$0+A = A$
2	Property of 1	$1.A = A$	$1+A = 1$
3	Idempotent Law	$A.A = A$	$A+A = A$
4	Inverse/Complement Law	$A.A' = 0$	$A+A' = 1$
5	Commutative Law	$A.B = B.A$	$A+B = B+A$
6	Associative Law	$A.(B.C) = (A.B).C$	$A+(B+C) = (A+B)+C$
7	Distributive Law	$A+BC = (A+B).(A+C)$	$A(B+C) = AB+AC$
8	Absorption Law	$A.(A+B) = A$	$A+AB = A$
9	De Morgan's Law	$\overline{A.B} = \bar{A} + \bar{B}$	$\overline{A+B} = \bar{A}.\bar{B}$

2.9 DE MORGAN'S LAW

A famous mathematician De Morgan has suggested two laws which are extremely useful in Boolean Algebra. The rules of De Morgan's theorem are produced from the Boolean expressions for OR, AND and NOT. The two laws have been discussed below.

The first law states that when two (or more) input variables are OR'ed and negated, they are equivalent to the AND of the complements of the individual variables.

$$\overline{A+B} = \bar{A} \cdot \bar{B}$$

Let $P = (A+B)$

Therefore,

$$P + P' = 1 \text{ and } P \cdot P' = 0$$

[Complementary law]

Substitute $(A+B)'$ for P and $(A+B)'$ for P' , we have:

$$(A+B) + (A+B)' = 1 \text{ and } (A+B) \cdot (A+B)' = 0$$

$$\text{But, } (A+B)' = A' \cdot B'$$

Therefore,

$$(A+B) + (A' \cdot B') = 1$$

and

$$(A+B) \cdot (A' \cdot B') = 0$$

$$\text{Let us prove } (A+B) + (A' \cdot B') = 1$$

$$= (A+B) + (A' \cdot B')$$

$$= (A+B+A') \cdot (A+B+B')$$

[Distributive Law]

$$= (A+A'+B) \cdot (A+B+B')$$

[Commutative law]

$$\text{We know that } A+A'=1 \text{ and } B+B'=1$$

[Inverse law]

Therefore,

$$= (1+B) \cdot (A+1)$$

[because, $1+A=1$ and $1+B=1$ using property of 1]

$$= 1 \cdot 1$$

$$= 1$$

Hence, proved.

$$\text{Let us prove } (A+B) \cdot (A' \cdot B') = 0$$

$$= (A+B) \cdot (\bar{A}' \cdot \bar{B}') = (A \cdot \bar{A} \cdot \bar{B}) + (B \cdot \bar{A} \cdot \bar{B})$$

[Distributive law]

$$(A \cdot A' \cdot B') + (A' \cdot B \cdot B')$$

[Commutative law]

$$= (0 \cdot B') + (A' \cdot 0)$$

[$A \cdot A' = 0$ and $B \cdot B' = 0$ Inverse law]

$$= 0 \cdot 0$$

[$0 \cdot A' = 0$ and $0 \cdot B' = 0$ property of 0]

$$= 0$$

Hence, proved.

Thus, we can say that $\overline{A+B} = \bar{A} \cdot \bar{B}$

Proof using truth table:

Table 2.5: Truth table

A	B	A'	B'	A+B	(A+B)'	A'.B'
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

Logic gate implementation of the above theorem is:

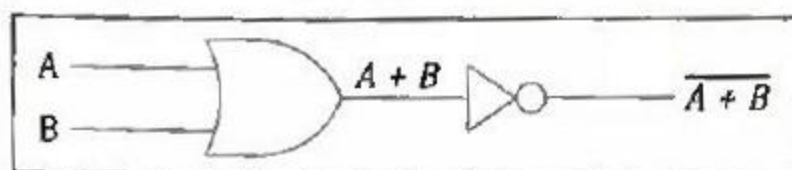
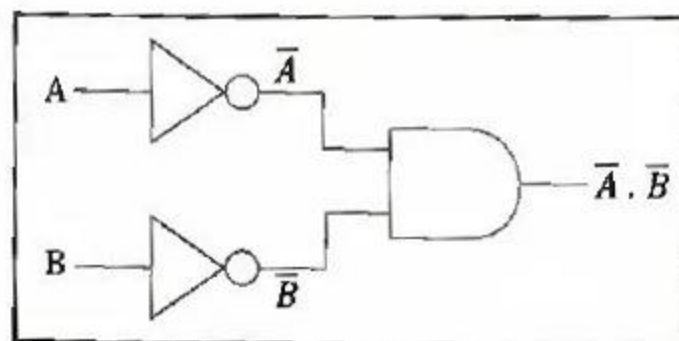


Fig. 2.6: First Law of De Morgan's Theorem

It is equivalent to:



The second law states that when two (or more) input variables are AND'ed and negated, they are equivalent to the OR of the complements of the individual variables.

$$\overline{A.B} = \bar{A} + \bar{B}$$

Let us prove it.

Let $P=A.B$ and $P'=(A.B)'$

Therefore, $P+P'=1$ and $P.P'=0$

$$(A.B) + (A.B)' = 1$$

$$= (A.B) + (A' + B')$$

$$= (A + \bar{A} + \bar{B}). (B + \bar{A} + \bar{B})$$

$$= (A + \bar{A} + \bar{B}). (\bar{A} + B + \bar{B})$$

$$= (1 + \bar{B}). (\bar{A} + 1)$$

$$= 1.1$$

$$= 1$$

[Distributive law]

[Commutative law]

[Inverse law]

Let us prove the second part $P.P'=0$

$$= (A.B).(A' + B')$$

$$= A.B.A' + A.B.B'$$

$$= A.A'.B + A.B.B'$$

$$= 0.B + A.0$$

$$= 0 + 0$$

$$= 0$$

[Distributive law]

[Commutative law]

[Inverse law $A.A'=0$]

[Property of 0, $0+0=0$]

Hence, proved.

Thus, we can say that:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

Proof using truth table:

Table 2.6: Truth table

A	B	A'	B'	A.B	(A.B)'	A+B'
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	0	0

Logic gate implementation of the above theorem is:

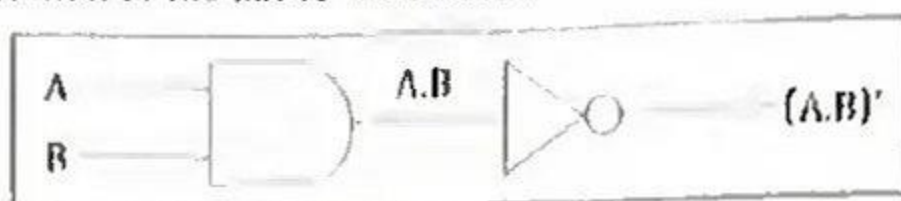
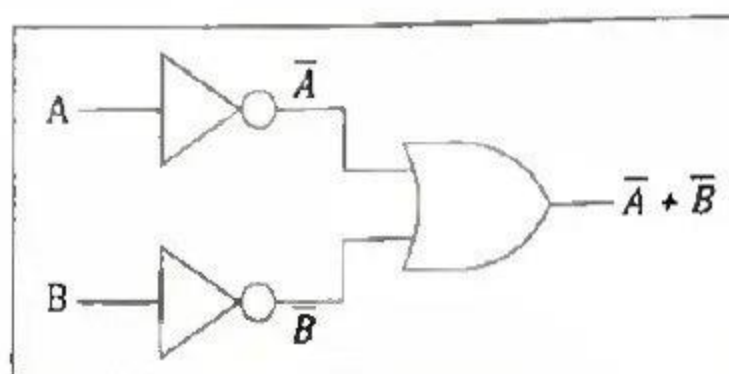


Fig. 2.7: Second Law of De Morgan's Theorem

It is equivalent to:



MEMORY BYTES

- Number system is the technique to represent numbers in the computer.
- Encoding scheme maps text into the codes that facilitate communication among computers.
- Textual data is encoded using ASCII, ISCII or Unicode.
- Unicode scheme is a character encoding standard which can encode all the characters of almost all language of the world.
- Computer being a digital system understands only binary numbers which are 0 and 1.
- Encoded text is converted into binary form for processing by the computer.
- Binary number system has only two symbols, and its base is 2.
- Base of a number is also known as radix of a number.
- The decimal numbers contain 10 digits or symbols : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- The octal numbers contain 8 digits: 0, 1, 2, 3, 4, 5, 6, 7
- The hexadecimal numbers contain 16 symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- The right-most digit is called the least significant bit and the left-most digit is called the most significant bit
- ASCII, ISCII and UNICODE are the three most popular encoding systems used today for character translations.
- Boolean statement is any logical statement which has a definite value which is either false or true
- A Boolean variable is also known as binary variable or logical variable that takes its values from the Boolean algebra.
- The values which are stored in binary variables are known as Boolean constants. For example, true/false yes/no or 1/0 are Boolean constants

- A Boolean variable is a variable which holds false/true value
- A Truth table is a representation of a Boolean function or expression containing all possible combinations of input values and their result in a tabular format
- Boolean operators are operators used in Boolean algebra. For example, AND, OR, NOT, NOR, NAND and XOR
- A Gate is a circuit which takes one or more inputs and generates an output
- Logic Gates use the binary operators AND, OR and NOT

OBJECTIVE TYPE QUESTIONS

1. Fill in the blanks.

- In binary number system, the left-most bit is called the significant bit
- Base of a number is also known as of a number.
- The decimal system is composed of symbols.
- code is the most widely used alphanumeric code which is used in computers to translate text (letters, numbers and symbols).
- Unicode is a new universal coding standard adopted by all new
- The operation is Boolean multiplication and the operation is Boolean addition.
- A statement is said to be a if it has a definite value, which is either True or False
- gate has only one input and it complements an input signal.
- The values which are stored in binary variables are known as
- A is a representation of a Boolean function or expression containing all possible combinations of input values and their result in a tabular format.

2. State whether the following statements are True or False.

- Binary data representation has only two symbols, 0 and 1.
- ASCII, ISCII and UNICODE are the three internal storage representations.
- $(128)_8$ is the representation of an octal number system.
- The first 128 characters are the same in both the types of ASCII, i.e., ASCII-7 and ASCII-8.
- 512 characters can be represented in ASCII-8.
- Unicode provides a consistent way of encoding multilingual plain text.
- ABC is a valid Hexadecimal number.
- A number with both integer and a fractional part has digits raised to both positive and negative powers of 2 in a decimal number system.
- $(52)_{16}$ is equivalent to 90 in Decimal number system.
- A number system that uses eight digits—0, 1, 2, 3, 4, 5, 6, and 7—is called an octal number system.
- NAND and NOR are called universal gates.

3. Multiple Choice Questions (MCQs):

- What is Information?

(i) Processed data	(ii) Collection of programs
(iii) Collection of instructions	(iv) None of these
- What is the base of binary number system?

(i) 8	(ii) 10	(iii) 2	(iv) 16
-------	---------	---------	---------
- What is the base of decimal number system?

(i) 2	(ii) 10	(iii) 8	(iv) 16
-------	---------	---------	---------
- What is the base of octal number system?

(i) 2	(ii) 10	(iii) 8	(iv) 16
-------	---------	---------	---------

- (e) Convert $(43)_{10}$ into binary number system:
 (i) $(101011)_2$ (ii) $(110011)_2$ (iii) $(110101)_2$ (iv) $(110010)_2$
- (f) Convert $(10010)_2$ into decimal number system:
 (i) 28 (ii) 18 (iii) 20 (iv) 22
- (g) What is the full form of ASCII?
 (i) International Standard Code for Information Interchange
 (ii) Indian Standard Code for Information Interchange
 (iii) International Script Code for Information Interchange
 (iv) None of these
- (h) Hexadecimal number system is composed of?
 (i) 2 symbols (ii) 10 symbols (iii) 8 symbols (iv) 16 symbols
- (i) Which of the following is not a binary number?
 (i) 1111 (ii) 101 (iii) 11E (iv) 000
- (j) Which numbering system uses numbers and letters as symbols?
 (i) Decimal (ii) Binary (iii) Octal (iv) Hexadecimal
- (k) Identify the logical statement:
 (i) $15+4=19$ (ii) I want to have coffee.
 (iii) Should I wear the mask or not? (iv) I love dogs.

SOLVED QUESTIONS

1. What do you understand by number system?

Ans. Number system is the technique to represent numbers in the computer system architecture. When the data is fed into the computer, it is converted into numeric format by ASCII (American Standard Code for Information Interchange), where each type of data is represented by its numerical equivalent. The value of each digit in such a number is determined by three things:

- The digit itself
- The position of the digit in the number
- The base or radix of the number

Examples of number systems are: Binary, octal, decimal and hexadecimal number systems.

2. Convert $(38.625)_{10}$ into its binary equivalent.

Ans. First convert 38 into a binary number, which is:

2	38	Remainder
2	19	0
2	9	1
2	4	1
2	2	0
2	1	0
2	0	1

Top
 ↑
 ↓
 Bottom

Hence, $(38)_{10} = (100110)_2$

Now, convert 0.625 into binary fraction, which is:

			Integer	Fraction
0.625×2	=	1.250	1	0.250
0.250×2	=	0.500	0	0.500
0.500×2	=	1.000	1	0.000
Therefore, $(0.625)_{10}$	=	$(0.101)_2$		
Hence, $(38.625)_{10}$	=	$(100110.101)_2$		

3. Convert $(1101)_2$ into decimal number.

Ans. Binary number is 1101.

$$\begin{aligned}\text{So, } 1101 &= (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ &= (1 \times 8) + (1 \times 4) + (0 \times 2) + (1 \times 1) \\ &= 8 + 4 + 0 + 1\end{aligned}$$

Answer is: $(13)_{10}$

4. Convert $(EF.B1)_{16} = (?)_{10}$

Ans. = E F . B 1

$$\begin{aligned}&= E \times 16^1 + F \times 16^0 + B \times 16^{-1} + 1 \times 16^{-2} \\ &= 14 \times 16 + 15 \times 1 + 11 \times (1/16) + 1 \times (1/256) \\ &= 224 + 15 + (0.6875) + (0.00390625) \\ &= 239 + 0.69140625 \\ &= 239.69140625\end{aligned}$$

Answer is: $(239.691406)_{10}$

5. Convert $(2C9)_{16}$ into decimal.

Ans. = $2 \times 16^2 + 12 \times 16^1 + 9 \times 16^0$

$$= 2 \times 256 + 12 \times 16 + 9 \times 1$$

$$= 512 + 192 + 9$$

$$= (713)_{10}$$

Answer is: $(713)_{10}$

6. Convert $(423)_{10}$ into hexadecimal.

Ans.

16	423	Remainder
16	26	7
16	1	10-A
	0	1

↑ Top
 ↓ Bottom

Now, write the remainders moving from MSB to LSB. We have $(1A7)_{16}$

Answer is: $(1A7)_{16}$

7. Do the following conversion:

(i) $(ABCD)_{16}$ to $(?)_2$

Ans. $(1010101111001101)_2$

(ii) $(3567)_{10}$ to $(?)_8$

Ans. $(6757)_8$

8. Draw a truth table and logic circuit diagram of NAND gate.

Ans.

A	B	$(AB)'$
0	0	1
0	1	1
1	0	1
1	1	0



9. Give the dual of $(A.1) + (B.(C+0))$.

Ans. $(A+0).(B+(C.1))$

10. The following is a message encoded in ASCII code. What is the message?

1001000 1000101 1001100 1010010

Ans. Convert each 7 bit code into its hexadecimal equivalent. The result is:

48 49 4C 50

Now locate these values in ASCII code table and determine the character represented by each. The result is
H I L P

11. What is Unicode? How is it useful?

Ans. Unicode is a new universal coding standard adopted by all new platforms. It is promoted by Unicode Consortium which is a non-profit organization. Unicode provides a unique number for every character irrespective of the platform, program and the language. It is a character coding system designed to support the worldwide interchange, processing and display of the written texts of diverse languages.

12. What are Boolean variables?

Ans. A Boolean variable is also known as binary variable or logical variable that takes its values from the Boolean algebra. A Boolean variable can take only one binary-valued quantity out of the two possible values, i.e., yes/no, 1/0, true/false. One bit represents one Boolean variable. In electrical circuit, if the signal passes, it represents 1, else it is represented through 0.

13. What are Boolean statements?

Ans. A statement is said to be a Boolean/Logical statement if it has a definite value which is either false or true.

14. What is truth table?

Ans. A truth table is a representation of a Boolean function or expression containing all possible combinations of input values and their results in a tabular format.

15. What are logic gates?

Ans. A Gate is a circuit which takes one or more inputs and generates an output. Logic Gates use the binary operators AND, OR and NOT.

16. Verify the following using truth table:

$$X + Y.Z = (X + Y).(X + Z)$$

Ans. $X + Y.Z = (X + Y).(X + Z)$

Truth Table

X	Y	Z	Y.Z	X+Y.Z	(X+Y)	(X+Z)	(X+Y).(X+Z)
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	0	0	1	1	1	1
1	1	1	1	1	1	1	1

The values for highlighted columns are the same. Hence, L.H.S. = R.H.S.

17. Verify the following using truth table:

(a) $X.X' = 0$

(b) $X+1 = 1$

Ans.

(a) $X.X' = 0$ (b) $X+1 = 1$

X	X'	X.X'	X+1
0	1	0	1
0	1	0	1
1	0	0	1
1	0	0	1

18. (a) Convert $(122)_{10}$ into binary number.

Ans.

2	122	Remainder
2	61	0
2	30	1
2	15	0
2	7	1
2	3	1
2	1	1
	0	1

Top
 ↑
 ↓
 Bottom

Therefore, $(122)_{10} = (1111010)_2$

(b) Convert $(122)_{10}$ into octal number.

Ans.

8	122	Remainder
8	15	2
8	1	7
	0	1

Top
 ↑
 ↓
 Bottom

Therefore, $(122)_{10} = (172)_8$

(c) Convert $(122)_{10}$ into hexadecimal number.

Ans.

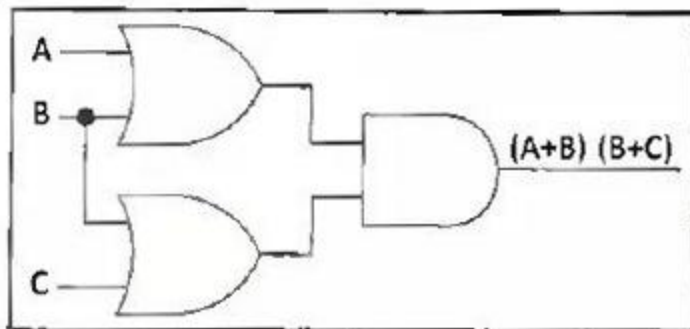
16	122	Remainder
16	7	10-A
	0	7

Top
 ↑
 ↓
 Bottom

Therefore, $(122)_{10} = (7A)_{16}$

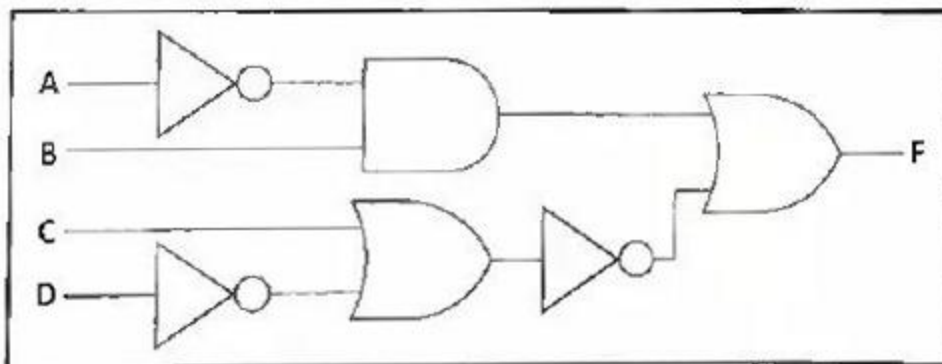
19. Draw a circuit diagram corresponding to the following Boolean expression:
 $(A + B)(B + C)$

Ans.



20. Obtain the Boolean expression for the logic circuit shown below:

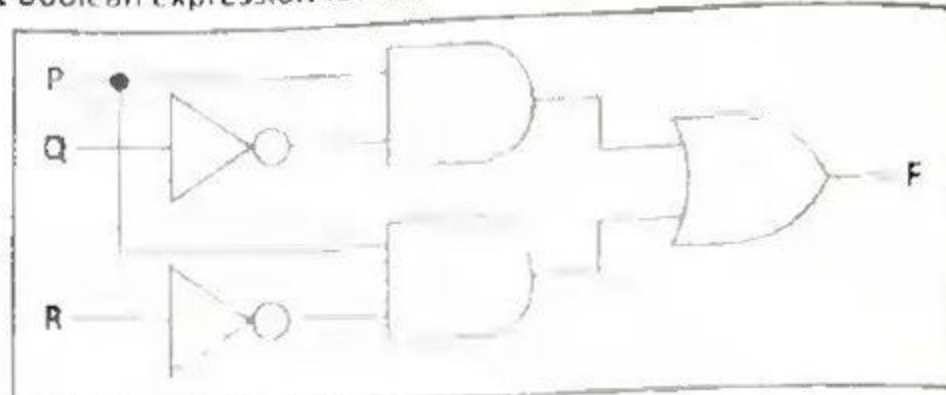
(CBSE D 2014)



Ans. $F = A' \cdot B + (C + D')$

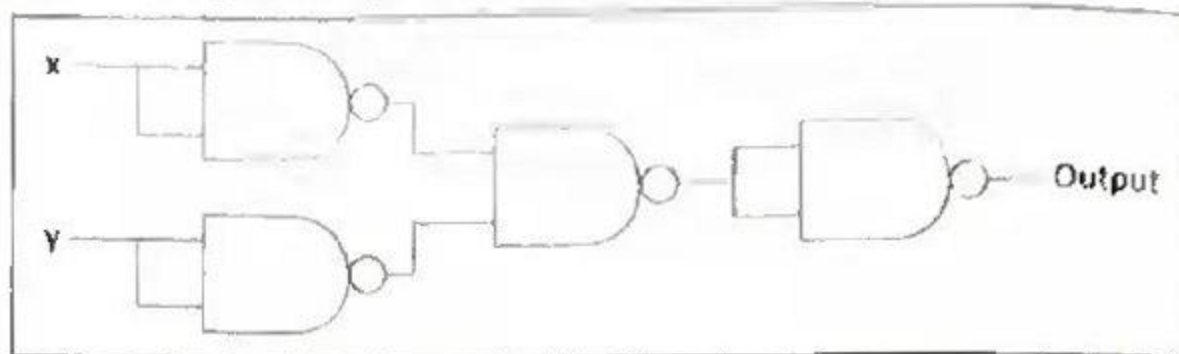
21. Write the equivalent Boolean expression for the following Logic Circuit:

(CBSE D 2r)



Ans. $F(P, Q, R) = P \cdot Q' + P \cdot R$

22. Which gate is the following circuit equivalent to?



- (a) AND (b) OR (c) NAND (d) NOR
(e) None of these

Ans. (d) NOR

23. Identify the logic gates that can implement the following Boolean expression:

- (i) $A \cdot B$ (ii) $X + Y \cdot Z'$ (iii) $PQR + \overline{QR}$

Ans. (i) $A \cdot B$ – It requires AND gate.

(ii) $X + Y \cdot Z'$ – It requires OR, AND and NOT gates.

(iii) $PQR + \overline{QR}$ – It requires OR, AND and NOT gates.

UNSOLVED QUESTIONS

1. Convert decimal number into binary number.

- (a) 52 (b) 44 (c) 25.80 (d) 62.325

2. Convert decimal number into octal number.

- (a) 911 (b) 540 (c) 61 (d) 132

3. Convert decimal number into hexadecimal number.

- (a) 132 (b) 3619 (c) 206 (d) 752

4. Convert binary number into decimal number.

- (a) 10111 (b) 111101 (c) 101010.011 (d) 101101

5. Convert octal number into decimal number.

- (a) 75 (b) 321 (c) 142 (d) 205

6. Convert hexadecimal number into decimal number.

- (a) A2 (b) 13B (c) 271 (d) 132

7. Discuss UTF-8 encoding scheme. How is it different from UTF-32 encoding scheme?

8. What are Boolean constants?

9. What are Boolean operators? Explain AND, OR, NOT operators.

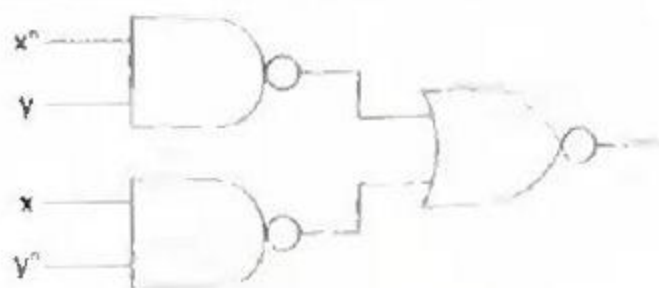
10. What is the use of Hexadecimal number system in computers?

11. Verify the following using truth table:

(i) $a + (a + b) = a$

(ii) $X \cdot (Y + Z) = X \cdot Y + X \cdot Z$

12. Obtain the Boolean expression for the logic circuit shown below:



13. State De Morgan's law and prove it with a truth table.

14. What is the advantage of preparing a digital content in Indian language using UNICODE font?

15. Encode the word 'COMPUTER' using ASCII and convert the encoded value into binary values.

16. Expand ASCII and ISCII.

17. Draw a logic circuit of the following Boolean expression:

(i) $(A \cdot B) + (C + \bar{D}) + (B \cdot \bar{D})$

(ii) $(A + B) \cdot (BC + \bar{D})$

18. Write binary equivalent of the following octal numbers:

(a) 2306

(b) 5610

(c) 742

(d) 65.203

19. Write binary representation of the following hexadecimal numbers:

(a) 4026

(b) BCA1

(c) 98E

(d) 132.45

20. Perform the following conversions from decimal number to other number systems:

(a) $(54)_{10} = (?)_2$

(b) $(120)_{10} = (?)_2$

(c) $(76)_{10} = (?)_8$

(d) $(889)_{10} = (?)_8$

(e) $(789)_{10} = (?)_{16}$

(f) $(108)_{10} = (?)_{16}$

21. Express the following binary numbers into their equivalent octal numbers:

(a) 100101111

(b) 111011010

(c) 1010011

(d) 10011101

22. Express the following binary numbers into their equivalent hexadecimal numbers:

(a) 1111101101100011

(b) 100000101011100

(c) 1000111010100011

(d) 111011111

23. Express the following octal numbers into their equivalent decimal numbers.

(a) 145

(b) 6760

(c) 455

(d) 10.75

24. The hexadecimal number system uses 16 literals (0 - 9, A - F). Write down its base value.

25. Explore and list the steps required to type in an Indian language using UNICODE.

26. Convert the following base of number system:

(a) $(1010100)_2 = (\dots\dots\dots)_{10}$

(b) $(3674)_8 = (\dots\dots\dots)_2$

(c) $(266)_{10} = (\dots\dots\dots)_8$

(d) $(9F2)_{16} = (\dots\dots\dots)_2$

27. Write the name of following law and verify using truth table:

$a(b + c) = ab + ac$

28. Draw the logic circuit for the following Boolean expression:

$(X' + Y) \cdot Z + W'$